

Importing libraries

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib
import matplotlib.pyplot as plt
```

```
print('Numpy Version',np.__version__)
print('Pandas Version',pd.__version__)
print('Seaborn Version',sns.__version__)
print('Matplotlib Version',matplotlib.__version__)
```

```
Numpy Version 2.0.2
Pandas Version 2.2.2
Seaborn Version 0.13.2
Matplotlib Version 3.10.0
```

Importing dataset

```
CC = pd.read_excel("/content/LoyaltyVision Analytics (1).xlsx")
CC.head()
```

	AccountID	Churn	Tenure	City_Tier	CC_Contacted_LY	Payment	Gender	Service_Score	Account_user_count	account_segment	CC_
0	20000	1	4	3.0	6.0	Debit Card	Female	3.0	3	Super	
1	20001	1	0	1.0	8.0	UPI	Male	3.0	4	Regular Plus	
2	20002	1	0	1.0	30.0	Debit Card	Male	2.0	4	Regular Plus	
3	20003	1	0	3.0	15.0	Debit Card	Male	2.0	4	Super	
4	20004	1	0	1.0	12.0	Credit Card	Male	2.0	3	Regular Plus	

Shape of dataset

```
## Shape of dataset
CC.shape
```

```
(11260, 19)
```

```
CC.size
```

```
213940
```

```
## High level information
CC.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11260 entries, 0 to 11259
Data columns (total 19 columns):
#   Column                Non-Null Count  Dtype
---  -
0   AccountID             11260 non-null  int64
1   Churn                 11260 non-null  int64
2   Tenure                11158 non-null  object
3   City_Tier             11148 non-null  float64
4   CC_Contacted_LY       11158 non-null  float64
5   Payment               11151 non-null  object
6   Gender                11152 non-null  object
7   Service_Score         11162 non-null  float64
8   Account_user_count    11148 non-null  object
9   account_segment       11163 non-null  object
10  CC_Agent_Score         11144 non-null  float64
11  Marital_Status         11048 non-null  object
12  rev_per_month          11158 non-null  object
13  Complain_ly            10903 non-null  float64
14  rev_growth_yoy         11260 non-null  object
15  coupon_used_for_payment 11260 non-null  object
16  Day_Since_CC_connect   10903 non-null  object
```

```

17  cashback          10789 non-null  object
18  Login_device      11039 non-null  object
dtypes: float64(5), int64(2), object(12)
memory usage: 1.6+ MB

```

Describing dataset

```
CC.describe()
```

	AccountID	Churn	City_Tier	CC_Contacted_LY	Service_Score	CC_Agent_Score	Complain_ly
count	11260.00000	11260.000000	11148.000000	11158.000000	11162.000000	11144.000000	10903.000000
mean	25629.50000	0.168384	1.653929	17.867091	2.902526	3.066493	0.285334
std	3250.62635	0.374223	0.915015	8.853269	0.725584	1.379772	0.451594
min	20000.00000	0.000000	1.000000	4.000000	0.000000	1.000000	0.000000
25%	22814.75000	0.000000	1.000000	11.000000	2.000000	2.000000	0.000000
50%	25629.50000	0.000000	1.000000	16.000000	3.000000	3.000000	0.000000
75%	28444.25000	0.000000	3.000000	23.000000	3.000000	4.000000	1.000000
max	31259.00000	1.000000	3.000000	132.000000	5.000000	5.000000	1.000000

Checking duplicates

```
CC.duplicated().sum()
```

```
np.int64(0)
```

Checking Null Values

```
CC.isnull().sum()
```

	0
AccountID	0
Churn	0
Tenure	102
City_Tier	112
CC_Contacted_LY	102
Payment	109
Gender	108
Service_Score	98
Account_user_count	112
account_segment	97
CC_Agent_Score	116
Marital_Status	212
rev_per_month	102
Complain_ly	357
rev_growth_yoy	0
coupon_used_for_payment	0
Day_Since_CC_connect	357
cashback	471
Login_device	221

```
dtype: int64
```

```

## Proportion of nulls in predictors out of total size of the predictor dataset expressed as percentage
CC_predictor = CC.drop(['AccountID', 'Churn'], axis=1)

round(((CC_predictor.isnull().sum().sum()/ CC_predictor.size) * 100 ),2)

```

```
np.float64(1.4)
```

```
## Nulls by rows
```

```
CC_predictor.isnull().sum(axis = 1)
```

	0
0	0
1	0
2	1
3	0
4	0
...	...
11255	0
11256	0
11257	0
11258	0
11259	0

11260 rows × 1 columns

dtype: int64

```
CC_predictor_temp = CC_predictor[CC_predictor.isnull().sum(axis = 1) >= 3]  
CC_predictor_temp
```

Tenure	City_Tier	CC_Contacted_LY	Payment	Gender	Service_Score	Account_user_count	account_segment	CC_Agent_Score	Marital_Status
--------	-----------	-----------------	---------	--------	---------------	--------------------	-----------------	----------------	----------------

```
CC_predictor_temp = CC_predictor[CC_predictor.isnull().sum(axis = 1) == 2]  
CC_predictor_temp.shape
```

(121, 17)

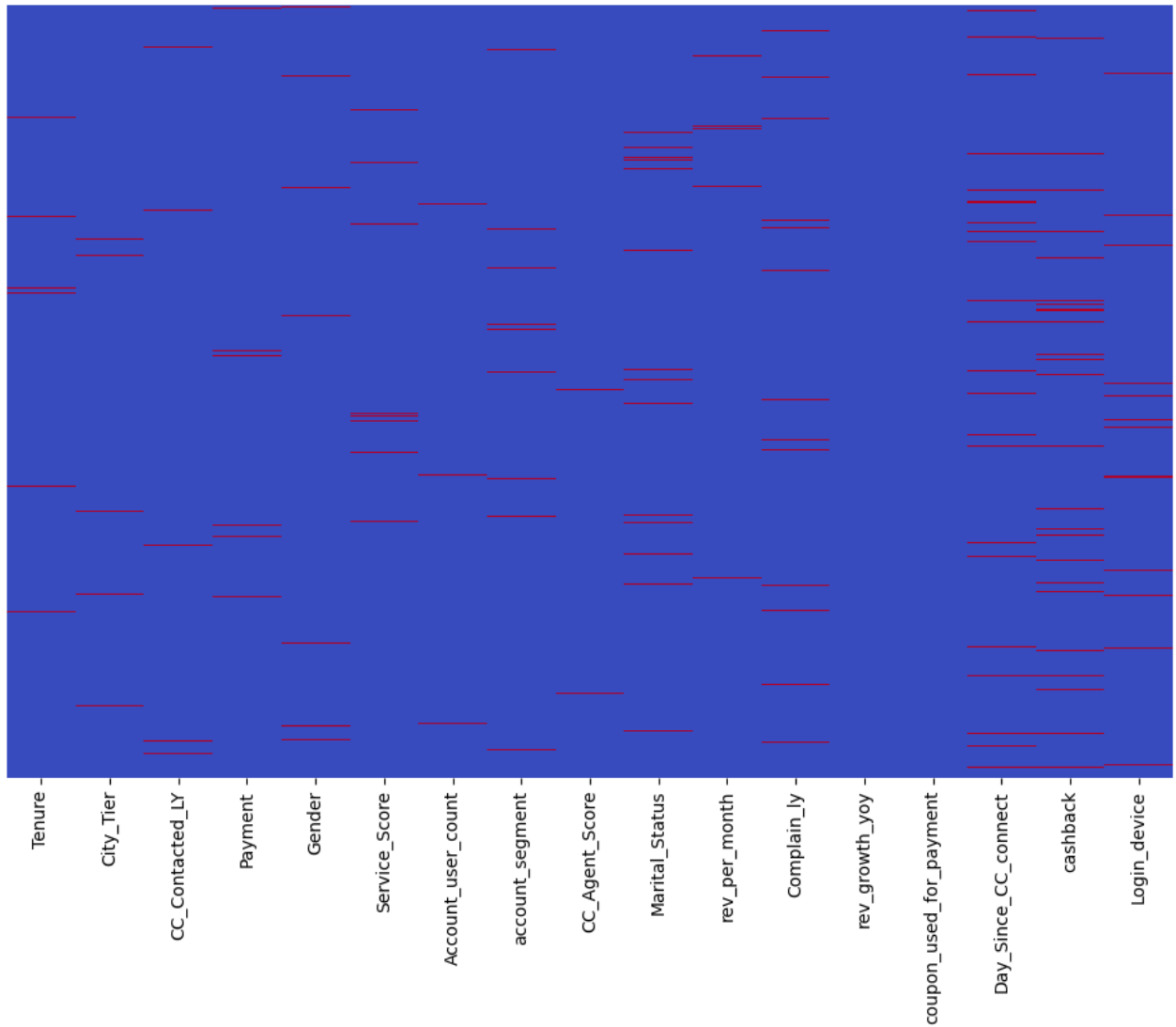
```
## Percentage nulls by column in predictor dataset
```

```
round((CC_predictor.isnull().sum().sort_values(ascending = False)/CC_predictor.index.size ) * 100, 2)
```

	0
cashback	4.18
Day_Since_CC_connect	3.17
Complain_ly	3.17
Login_device	1.96
Marital_Status	1.88
CC_Agent_Score	1.03
Account_user_count	0.99
City_Tier	0.99
Payment	0.97
Gender	0.96
Tenure	0.91
CC_Contacted_LY	0.91
rev_per_month	0.91
Service_Score	0.87
account_segment	0.86
rev_growth_yoy	0.00
coupon_used_for_payment	0.00

dtype: float64

```
plt.figure(figsize = (12,8))  
sns.heatmap(CC_predictor.isnull(), cbar = False, cmap = 'coolwarm', yticklabels = False)  
plt.show()
```



▼ Data Cleaning

```
## Tenure field  
CC.Tenure.value_counts()
```

count	
Tenure	
1	1351
0	1231
8	519
9	496
7	450
10	423
3	410
4	403
5	403
11	388
6	363
12	360
13	359
2	354
14	345
15	311
16	291
19	273
18	253
20	217
17	215
21	170
23	169
22	151
24	147
28	137
30	137
99	131
27	131
26	122

##Replacing # in tenure with nulls. There are 116 rows where Tenure has the value '#'. Filling this with np.nan

```
CC['Tenure'] = CC['Tenure'].replace('#',np.nan)
```

```
<ipython-input-17-d38ce7d15720>:3: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version of pandas.
CC['Tenure'] = CC['Tenure'].replace('#',np.nan)
```

```
CC.Tenure.value_counts(dropna=False)
```

51	2
61	2

dtype: int64

count	
Tenure	
1.0	1351
0.0	1231
8.0	519
9.0	496
7.0	450
10.0	423
3.0	410
4.0	403
5.0	403
11.0	388
6.0	363
12.0	360
13.0	359
2.0	354
14.0	345
15.0	311
16.0	291
19.0	273
18.0	253
NaN	218
20.0	217
17.0	215
21.0	170
23.0	169
22.0	151
24.0	147
28.0	137
30.0	137
99.0	131
27.0	131

```
## Checking for City_tier column
CC.City_Tier.value_counts(dropna=False)
```

count	
City_Tier	
31.0	96
50.0	2
1.0	7263
3.0	3405
51.0	2
2.0	480
NaN	112
dtype: int64	
dtype: int64	

```
## Checking for CC_Contacted_LY column
CC.CC_Contacted_LY.value_counts(dropna=False)
```

CC_Contacted_LY	count
14.0	682
16.0	663
9.0	655
13.0	655
15.0	623
12.0	571
8.0	538
17.0	525
11.0	524
10.0	489
7.0	391
18.0	374
19.0	364
20.0	319
6.0	311
21.0	310
22.0	282
23.0	241
24.0	214
25.0	197
32.0	192
29.0	181
28.0	178
34.0	178
30.0	175
27.0	174
26.0	169
31.0	165
35.0	165
33.0	155

```
## Checking for payment
CC.Payment.value_counts(dropna=False)
```

Payment	count
38.0	73
39.0	55
Debit Card	4587
Credit Card	3511
42.0	30
E wallet	1217
Cash on Delivery	1014
43.0	8
UPI	822
NaN	109
4.0	1
dtype: int64	

```
## Checking for Gender
CC.Gender.value_counts(dropna=False)

dtype: int64
```

```
count
Gender
Male    6328
Female  4178
M        376
F        270
NaN      108
```

dtype: int64

```
CC['Gender'] = CC['Gender'].replace('M', 'Male')
```

```
CC['Gender'] = CC['Gender'].replace('F', 'Female')
```

```
CC.Gender.value_counts(dropna=False)
```

```
count
Gender
Male    6704
Female  4448
NaN      108
```

dtype: int64

```
## Checking for Service_Score
```

```
CC.Service_Score.value_counts(dropna=False)
```

```
count
Service_Score
3.0    5490
2.0    3251
4.0    2331
NaN      98
1.0      77
0.0       8
5.0       5
```

dtype: int64

```
## Checking for Account_user_count
```

```
CC.Account_user_count.value_counts(dropna=False)
```

```
count
Account_user_count
4    4569
3    3261
5    1699
2     526
1     446
@     332
6     315
NaN    112
```

dtype: int64

```
CC['Account_user_count'] = CC['Account_user_count'].replace('@', np.nan)
```

```
<ipython-input-28-e53145c0dba3>:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version of pandas. To suppress this warning, please replace `CC['Account_user_count'] = CC['Account_user_count'].replace('@', np.nan)` with `CC['Account_user_count'] = CC['Account_user_count'].replace('@', np.nan, dtype=object)`
```

```
CC.Account_user_count.value_counts(dropna=False)
```

	count
Account_user_count	
4.0	4569
3.0	3261
5.0	1699
2.0	526
1.0	446
NaN	444
6.0	315

```
dtype: int64
```

```
## Checking for account_segment
```

```
CC.account_segment.value_counts(dropna=False)
```

	count
account_segment	
Super	4062
Regular Plus	3862
HNI	1639
Super Plus	771
Regular	520
Regular +	262
NaN	97
Super +	47

```
dtype: int64
```

```
CC['account_segment'] = CC['account_segment'].replace( {'Regular +': 'Regular_Plus', 'Super +': 'Super_Plus', 'Super Plus': 'Super_Plus'})
```

```
CC.account_segment.value_counts(dropna=False)
```

	count
account_segment	
Regular_Plus	4124
Super	4062
HNI	1639
Super_Plus	818
Regular	520
NaN	97

```
dtype: int64
```

```
## Checking for CC_Agent_Score
```

```
CC.CC_Agent_Score.value_counts(dropna=False)
```

	count
CC_Agent_Score	
3.0	3360
1.0	2302
5.0	2191
4.0	2127
2.0	1164
NaN	116

dtype: int64

```
## Checking for Marital_Status

CC.Marital_Status.value_counts(dropna=False)
```

	count
Marital_Status	
Married	5860
Single	3520
Divorced	1668
NaN	212

dtype: int64

```
## Checking for rev_per_month

CC.rev_per_month.value_counts(dropna=False)
```

	count
rev_per_month	
3	1746
2	1585
5	1337
4	1218
6	1085
7	754
+	689
8	643
9	564
10	413
1	402
11	278
12	166
NaN	102
13	93
14	48
15	24
102	8
124	5
123	5
107	5
133	4
140	4
129	4
118	4
136	4
127	3
130	3
115	3
116	3

```
CC['rev_per_month'] = CC['rev_per_month'].replace('+',np.nan)
```

```
<ipython input-36-2e35bcf8e736>:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version of pandas.  
CC['rev_per_month'] = CC['rev_per_month'].replace('+',np.nan)  
101      3
```

```
## Checking for Complain_ly  
  
CC.Complain_ly.value_counts(dropna=False)
```

	count
Complain_ly	
0.0	7792
1.0	3111
NaN	357
19	2
dtype: int64	
134	2

```
## Checking for rev_growth_yoy  
  
CC.rev_growth_yoy.value_counts(dropna=False)  
  
105      2  
  
121      2  
  
22      2
```

125	count
rev_growth_yoy	1
14	1524
123	1427
15	1283
104	1210
16	949
135	708
17	704
109	619
20	562
103	523
21	433
22	403
23	345
24	229
25	188
26	98
27	35
28	14
4	3
\$	3

dtype: int64

```
CC['rev_growth_yoy'] = CC['rev_growth_yoy'].replace('$',np.nan)
```

```
<ipython-input-39-eb14f698bf41>:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version.
CC['rev_growth_yoy'] = CC['rev_growth_yoy'].replace('$',np.nan)
```

```
CC.rev_growth_yoy.value_counts(dropna=False)
```

	count
rev_growth_yoy	
14.0	1524
13.0	1427
15.0	1283
12.0	1210
16.0	949
18.0	708
17.0	704
19.0	619
20.0	562
11.0	523
21.0	433
22.0	403
23.0	345
24.0	229
25.0	188
26.0	98
27.0	35
28.0	14
4.0	3
NaN	3

dtype: int64

```
## Checking for coupon_used_for_payment
```

```
CC.coupon_used_for_payment.value_counts(dropna=False)
```

	count
coupon_used_for_payment	
1	4373
2	2656
0	2150
3	698
4	424
5	284
6	234
7	184
8	88
10	34
9	34
11	30
12	26
13	22
14	12
15	4
16	4
#	1
\$	1
*	1

dtype: int64

```
CC['coupon_used_for_payment'] = CC['coupon_used_for_payment'].replace( {'#': np.nan, '*': np.nan, '$': np.nan} )
```

```
<ipython-input-42-b5c2985ce588>:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version.
CC['coupon_used_for_payment'] = CC['coupon_used_for_payment'].replace( {'#': np.nan, '*': np.nan, '$': np.nan} )
```

```
CC.coupon_used_for_payment.value_counts(dropna=False)
```

	count
coupon_used_for_payment	
1.0	4373
2.0	2656
0.0	2150
3.0	698
4.0	424
5.0	284
6.0	234
7.0	184
8.0	88
9.0	34
10.0	34
11.0	30
12.0	26
13.0	22
14.0	12
15.0	4
16.0	4
NaN	3

dtype: int64

```
## Checking for Day_Since_CC_connect
```

```
CC.Day_Since_CC_connect.value_counts(dropna=False)
```

	count
Day_Since_CC_connect	
3	1816
2	1574
1	1256
8	1169
0	964
7	911
4	893
9	622
5	479
NaN	357
10	339
6	229
11	183
12	146
13	117
14	74
15	37
17	34
16	26
18	26
30	2
47	2
31	2
\$	1
46	1

dtype: int64

```
CC['Day_Since_CC_connect'] = CC['Day_Since_CC_connect'].replace('$',np.nan)
```

```
<ipython-input-45-26f1cb3c0abf>:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version.
CC['Day_Since_CC_connect'] = CC['Day_Since_CC_connect'].replace('$',np.nan)
```

```
CC.Day_Since_CC_connect.value_counts(dropna=False)
```

	count
Day_Since_CC_connect	
3.0	1816
2.0	1574
1.0	1256
8.0	1169
0.0	964
7.0	911
4.0	893
9.0	622
5.0	479
NaN	358
10.0	339
6.0	229
11.0	183
12.0	146
13.0	117
14.0	74
15.0	37
17.0	34
16.0	26
18.0	26
30.0	2
31.0	2
47.0	2
46.0	1

dtype: int64

Checking for cashback

CC.cashback.value_counts(dropna=False)

	count
cashback	
NaN	471
155.62	10
149.68	9
149.36	9
154.73	9
...	...
235.22	1
262.06	1
177.98	1
158.6	1
251.57	1

5694 rows × 1 columns

dtype: int64

CC[CC['cashback'] == '\$']

	AccountID	Churn	Tenure	City_Tier	CC_Contacted_LY	Payment	Gender	Service_Score	Account_user_count	account_segment
440	20440	0	0.0	1.0	16.0	Credit Card	Male	3.0	2.0	Regular_Plus
1257	21257	0	4.0	1.0	9.0	Credit Card	Male	2.0	3.0	Regular_Plus

```
CC['cashback'] = CC['cashback'].replace('$',np.nan)

<ipython-input-49-3fd185bc7693>:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version.
CC['cashback'] = CC['cashback'].replace('$',np.nan)
```

```
CC.cashback.dtype

dtype('float64')
```

```
## Checking for Login_device

CC.Login_device.value_counts(dropna=False)
```

	count
Login_device	
Mobile	7482
Computer	3018
&&&&	539
NaN	221

```
dtype: int64
```

```
CC['Login_device'] = CC['Login_device'].replace('&&&&',np.nan)
```

```
CC.Login_device.value_counts(dropna=False)
```

	count
Login_device	
Mobile	7482
Computer	3018
NaN	760

```
dtype: int64
```

After Data Cleaning

```
CC.drop("AccountID", axis=1).describe().T
```

	count	mean	std	min	25%	50%	75%	max
Churn	11260.0	0.168384	0.374223	0.0	0.00	0.00	0.00	1.0
Tenure	11042.0	11.025086	12.879782	0.0	2.00	9.00	16.00	99.0
City_Tier	11148.0	1.653929	0.915015	1.0	1.00	1.00	3.00	3.0
CC_Contacted_LY	11158.0	17.867091	8.853269	4.0	11.00	16.00	23.00	132.0
Service_Score	11162.0	2.902526	0.725584	0.0	2.00	3.00	3.00	5.0
Account_user_count	10816.0	3.692862	1.022976	1.0	3.00	4.00	4.00	6.0
CC_Agent_Score	11144.0	3.066493	1.379772	1.0	2.00	3.00	4.00	5.0
rev_per_month	10469.0	6.362594	11.909686	1.0	3.00	5.00	7.00	140.0
Complain_ly	10903.0	0.285334	0.451594	0.0	0.00	0.00	1.00	1.0
rev_growth_yoy	11257.0	16.193391	3.757721	4.0	13.00	15.00	19.00	28.0
coupon_used_for_payment	11257.0	1.790619	1.969551	0.0	1.00	1.00	2.00	16.0
Day_Since_CC_connect	10902.0	4.633187	3.697637	0.0	2.00	3.00	8.00	47.0
cashback	10787.0	196.236370	178.660514	0.0	147.21	165.25	200.01	1997.0

✓ After data cleaning null check

```
CC_predictor = CC.drop(['AccountID','Churn'],axis=1)

print("\nTotal number of nulls in all predictor fields ")
print(CC_predictor.isnull().sum().sum())

print("\nTotal % nulls in all predictor fields ")
print(round(((CC_predictor.isnull().sum().sum())/ CC_predictor.size) * 100 ),2))

## Percentage nulls by column in predictor dataset
print("\n\nProportion of nulls by column - multiply by 100 for percentage")
print(CC_predictor.isnull().sum().sort_values(ascending = False)/CC_predictor.index.size)

## Rows that have more than 4 null fields
print("\n\nRows that have more than 4 null fields")
CC_predictor_temp = CC_predictor[CC_predictor.isnull().sum(axis = 1) >= 4]
CC_predictor_temp
```

Total number of nulls in all predictor fields
4361

Total % nulls in all predictor fields
2.28

Proportion of nulls by column - multiply by 100 for percentage

```
rev_per_month      0.070249
Login_device       0.067496
cashback           0.042007
Account_user_count 0.039432
Day_Since_CC_connect 0.031794
Complain_ly        0.031705
Tenure             0.019361
Marital_Status     0.018828
CC_Agent_Score     0.010302
City_Tier          0.009947
Payment            0.009680
Gender             0.009591
CC_Contacted_LY    0.009059
Service_Score      0.008703
account_segment    0.008615
rev_growth_yoy     0.000266
coupon_used_for_payment 0.000266
dtype: float64
```

Rows that have more than 4 null fields

Tenure	City_Tier	CC_Contacted_LY	Payment	Gender	Service_Score	Account_user_count	account_segment	CC_Agent_Score	Marita
--------	-----------	-----------------	---------	--------	---------------	--------------------	-----------------	----------------	--------

✓ Outlier checking

```
## Making a separate list for numeric and categorical columns
```

```
num_cols = ['Tenure', 'CC_Contacted_LY', 'rev_per_month', 'rev_growth_yoy', 'coupon_used_for_payment', 'Day_Since_CC_connect', 'cas  
cat_cols = ['City_Tier', 'Payment', 'Gender', 'Service_Score', 'Account_user_count', 'account_segment', 'CC_Agent_Score', 'Marital_Sta
```

```
## Get upper bound and lower bound  
Q1 = CC_predictor[num_cols].quantile(0.25)  
Q3 = CC_predictor[num_cols].quantile(0.75)  
IQR = Q3 - Q1  
UL = Q3 + 1.5*IQR  
LL = Q1 - 1.5*IQR
```

```
print ("Proportion of outliers in numeric continuous columns \n")  
(((CC_predictor[num_cols] > UL) | (CC_predictor[num_cols] < LL)).sum())/ CC_predictor.shape[0]
```

Proportion of outliers in numeric continuous columns

	0
Tenure	0.012345
CC_Contacted_LY	0.003730
rev_per_month	0.016430
rev_growth_yoy	0.000000
coupon_used_for_payment	0.122558
Day_Since_CC_connect	0.002931
cashback	0.078064

dtype: float64

```
print ("Number of outliers in numeric continuous columns \n")  
((CC_predictor[num_cols] > UL) | (CC_predictor[num_cols] < LL)).sum().sum()
```

Number of outliers in numeric continuous columns

np.int64(2658)

```
print ("Proportion of outliers in all numeric continuous columns \n")  
((CC_predictor[num_cols] > UL) | (CC_predictor[num_cols] < LL)).sum().sum()/(CC_predictor.shape[0]*CC_predictor.shape[1])
```

Proportion of outliers in all numeric continuous columns

np.float64(0.013885696374464528)

CC_predictor.shape

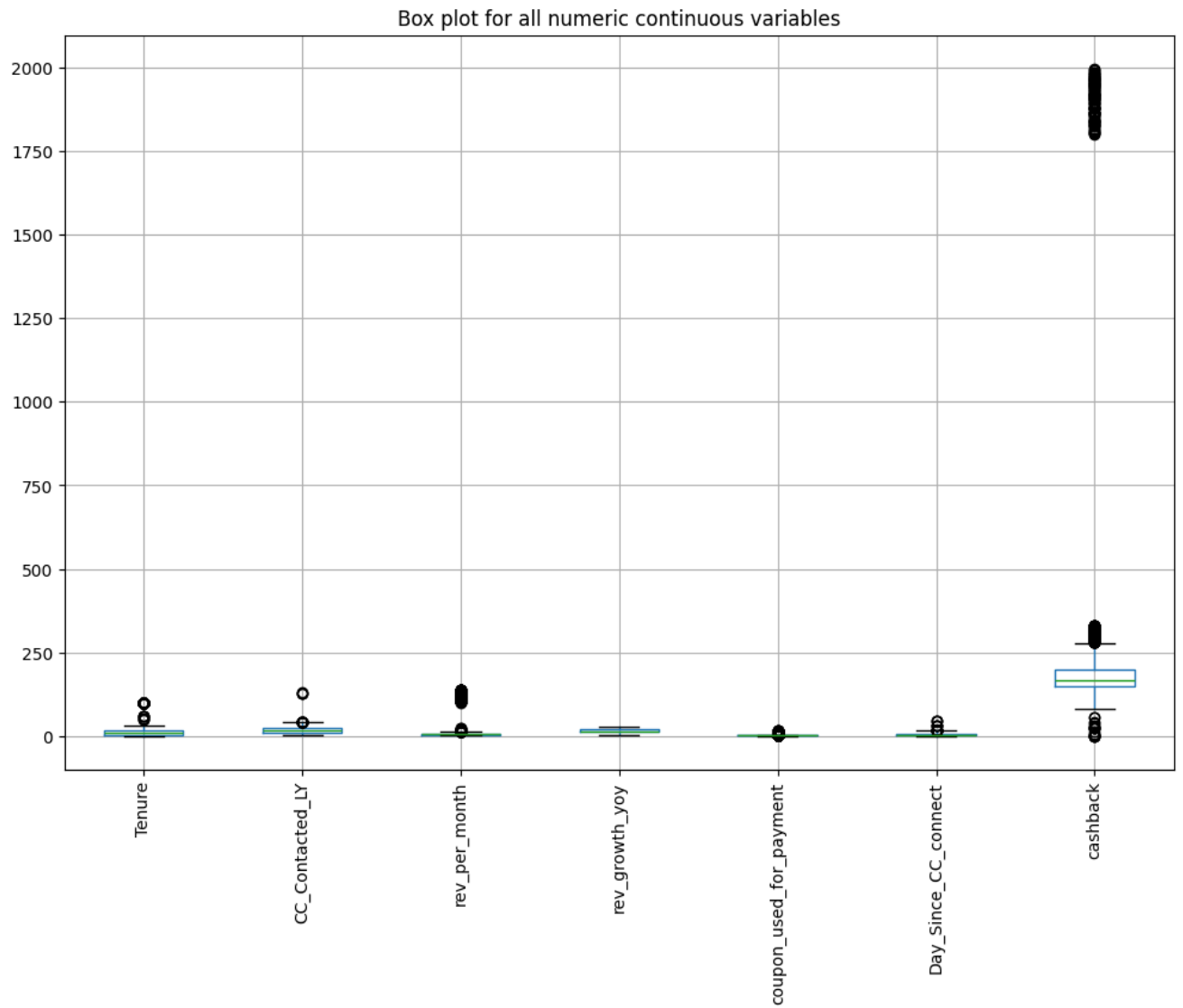
(11260, 17)

✓ Univariate Analysis

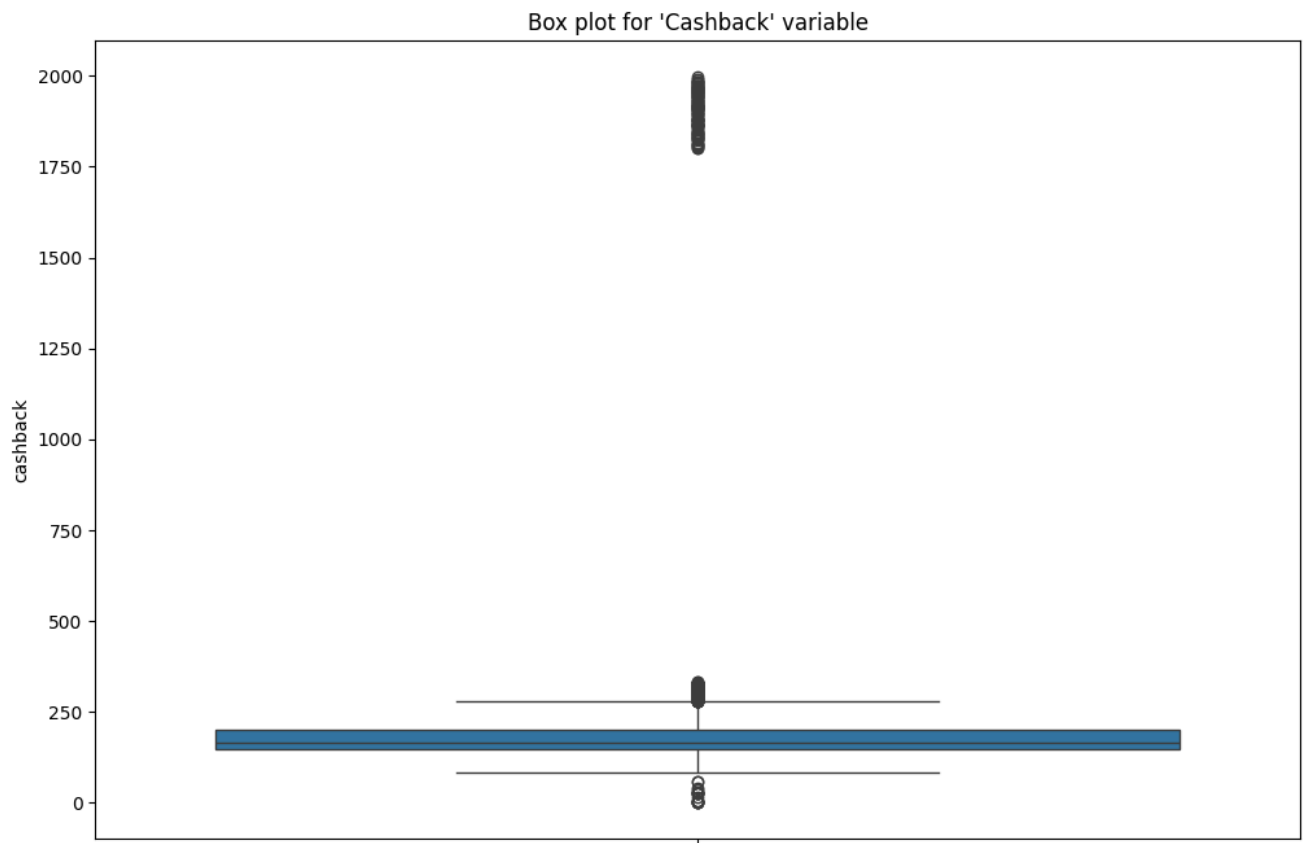
✓ Univariate Analysis for continuous data

✓ Box plot for continuous variables

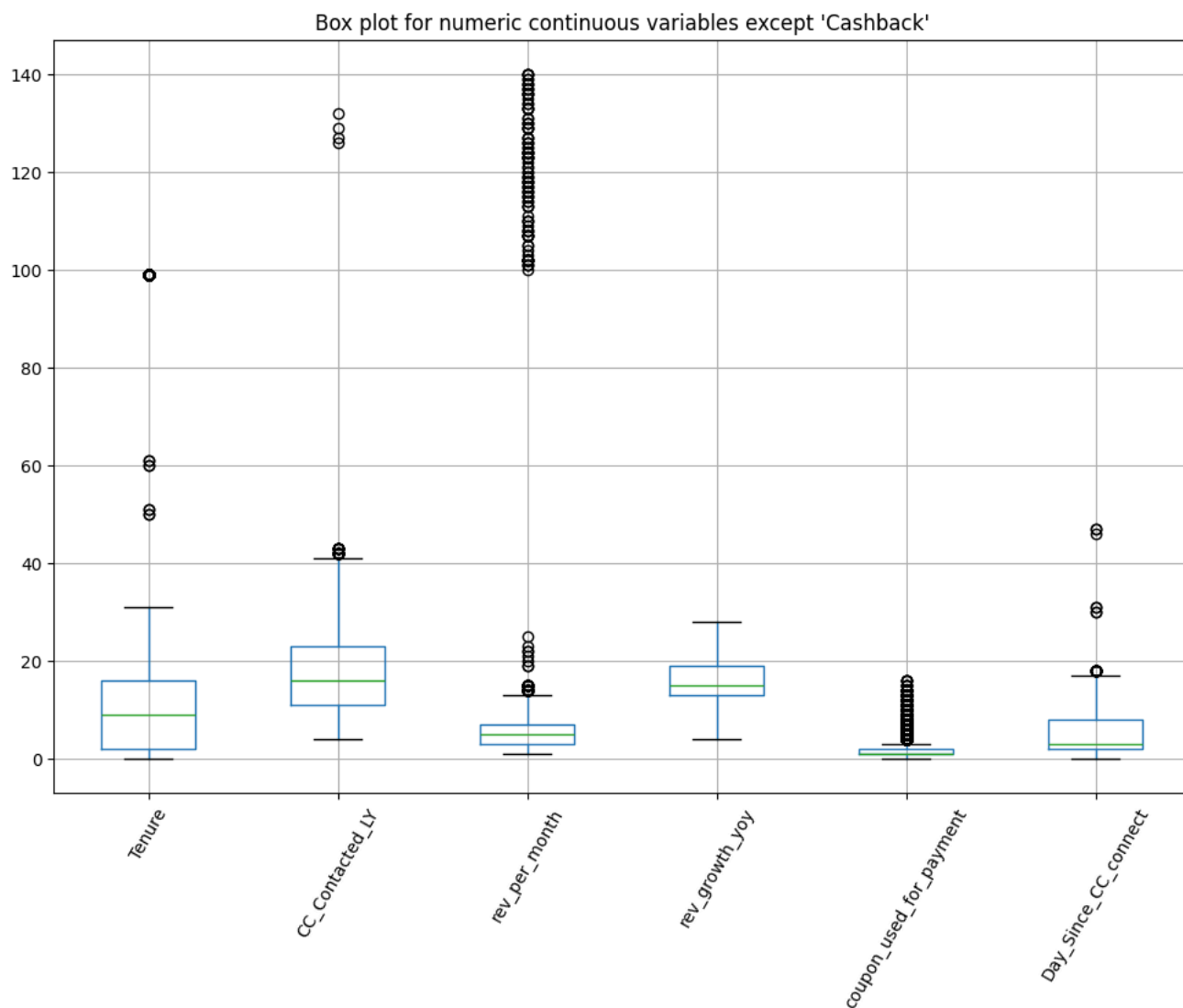
```
plt.figure(figsize=(12,8))  
CC_predictor[num_cols].boxplot()  
plt.xticks(rotation=90)  
plt.title("Box plot for all numeric continuous variables");
```



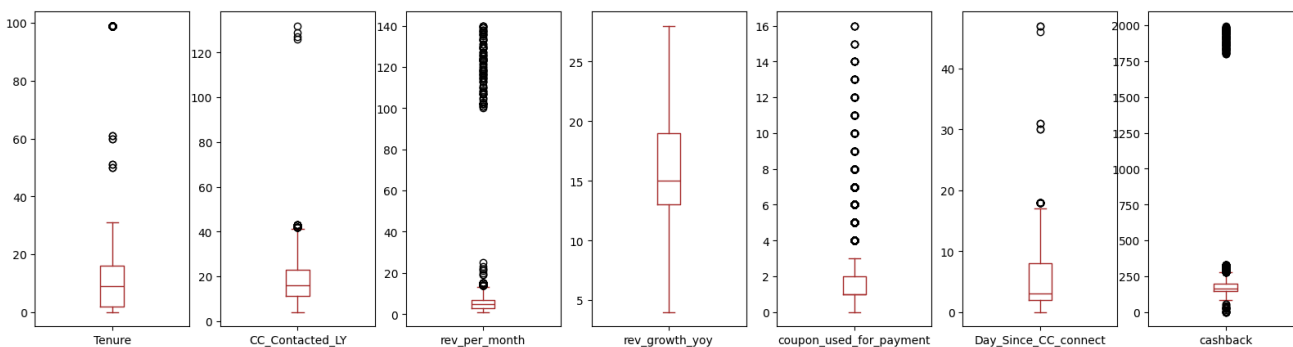
```
plt.figure(figsize=(12,8))
sns.boxplot(data=CC, y='cashback')
plt.xticks(rotation=90)
plt.title("Box plot for 'Cashback' variable");
```



```
plt.figure(figsize=(12,8))
CC_predictor[num_cols].drop('cashback',axis=1).boxplot()
plt.xticks(rotation=60)
plt.title("Box plot for numeric continuous variables except 'Cashback'");
```



```
CC[num_cols].plot(kind = 'box', subplots = True,
    figsize = (20,5), color = 'brown');
plt.title("Boxplot for numeric columns");
```



```
CC['coupon_used_for_payment'].value_counts()
```

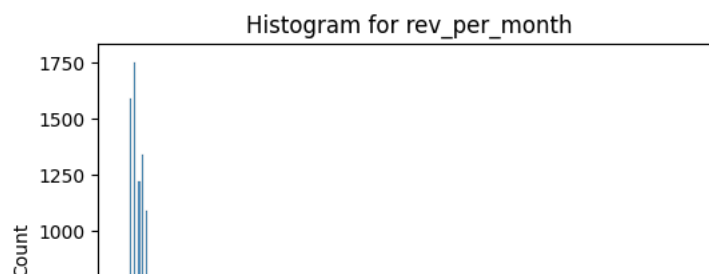
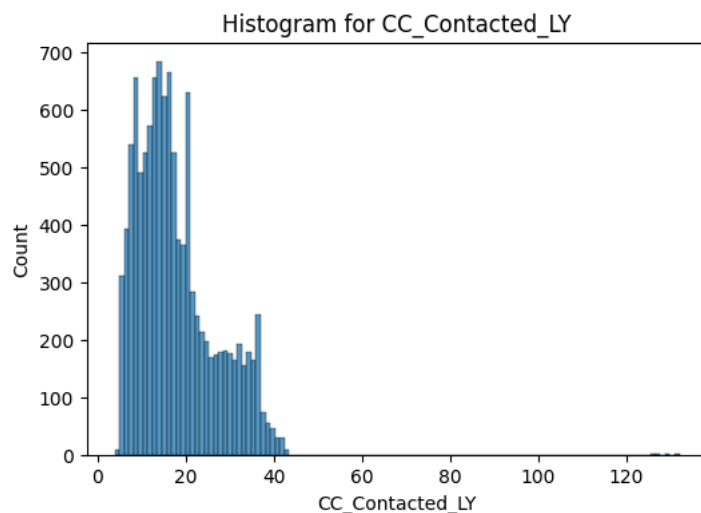
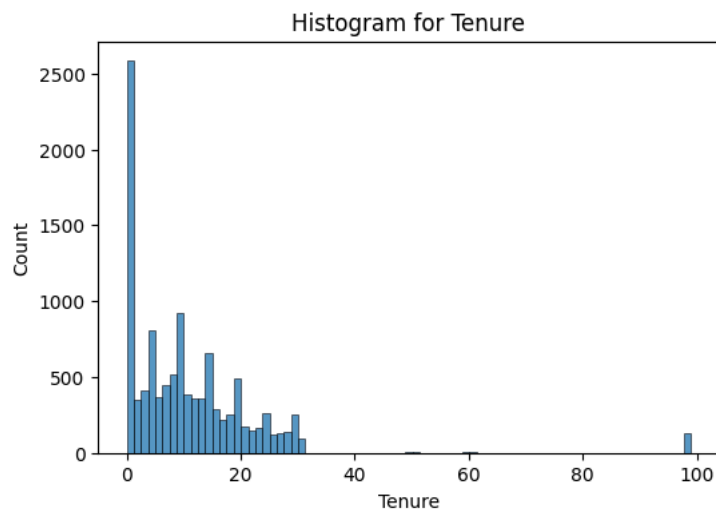
	count
coupon_used_for_payment	
1.0	4373
2.0	2656
0.0	2150
3.0	698
4.0	424
5.0	284
6.0	234
7.0	184
8.0	88
9.0	34
10.0	34
11.0	30
12.0	26
13.0	22
14.0	12
15.0	4
16.0	4

dtype: int64

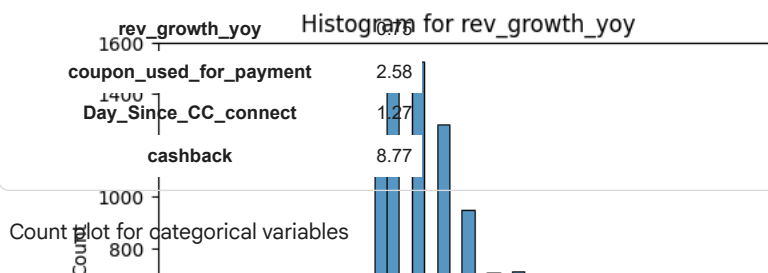
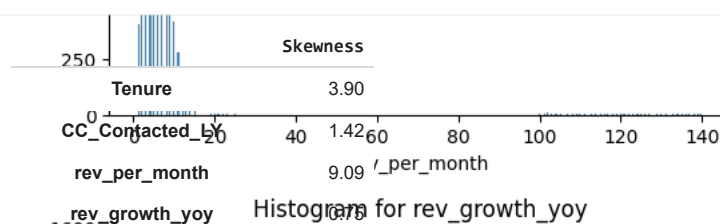
```
Outlier_treat_vars = ['Tenure', 'CC_Contacted_LY', 'rev_per_month', 'Day_Since_CC_connect', 'cashback']
```

▼ Histogram for continuous variables

```
## Using histplot
for i in num_cols:
    plt.figure(figsize=(6,4))
    sns.histplot(data=CC, x=i)
    plot_title = 'Histogram for ' + i
    plt.title(plot_title)
    plt.show();
```



```
cols = ['Skewness']
pd.DataFrame(CC[num_cols].skew(skipna=True).round(2), columns=cols)
```

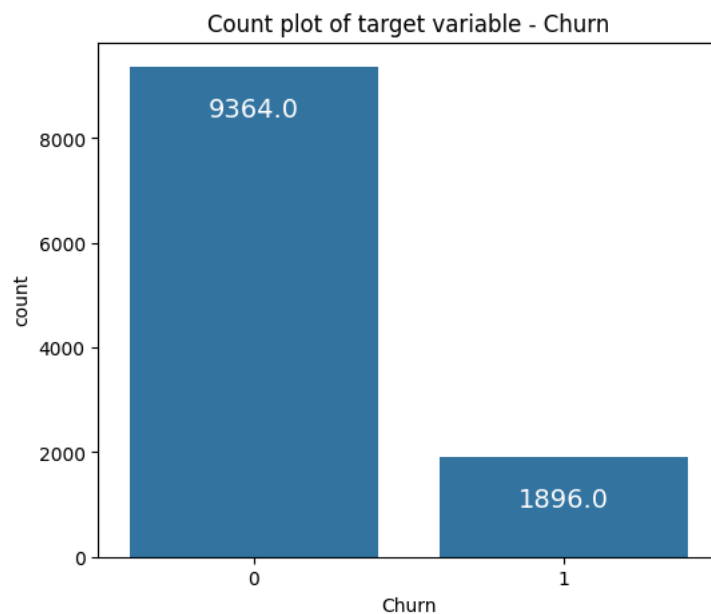


```
## Count plot of target column
```

```
plt.figure(figsize=(6,5))
ax = sns.countplot(x='Churn', data=CC)
```

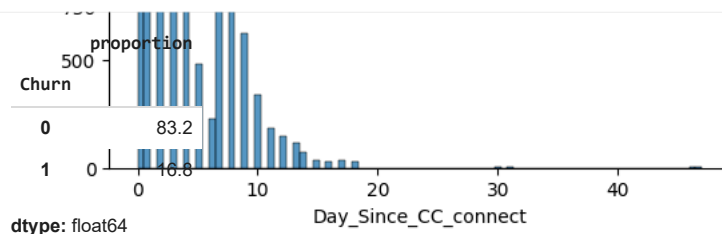
```
for p in ax.patches:
    ax.annotate(f'\n{p.get_height()}', (p.get_x()+0.4, p.get_height()), ha='center', va='top', color='white', size=14)
```

```
plt.title("Count plot of target variable - Churn");
```



```
## Checking for balance of target variable in the dataset
```

```
(CC['Churn'].value_counts(normalize=True).round(3)) * 100
```



```
dtype: float64
```

Histogram for cashback

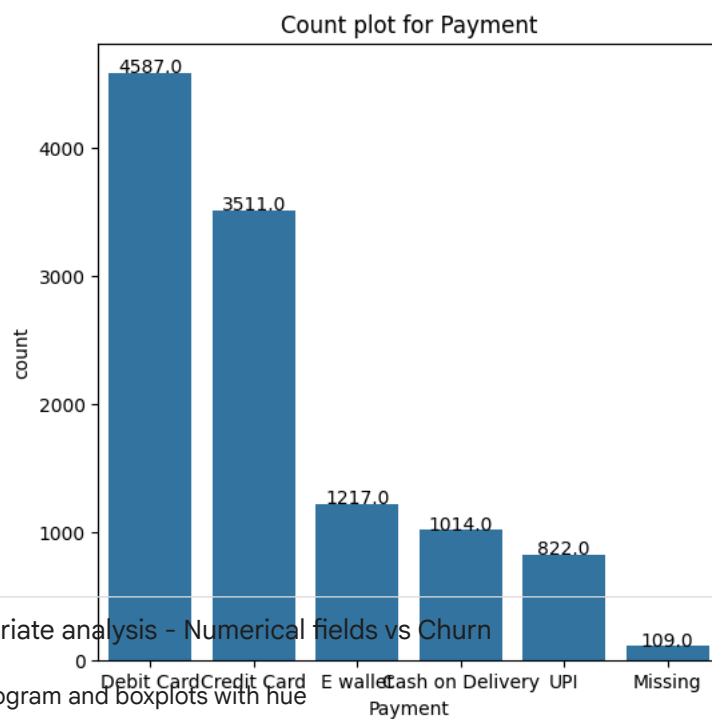
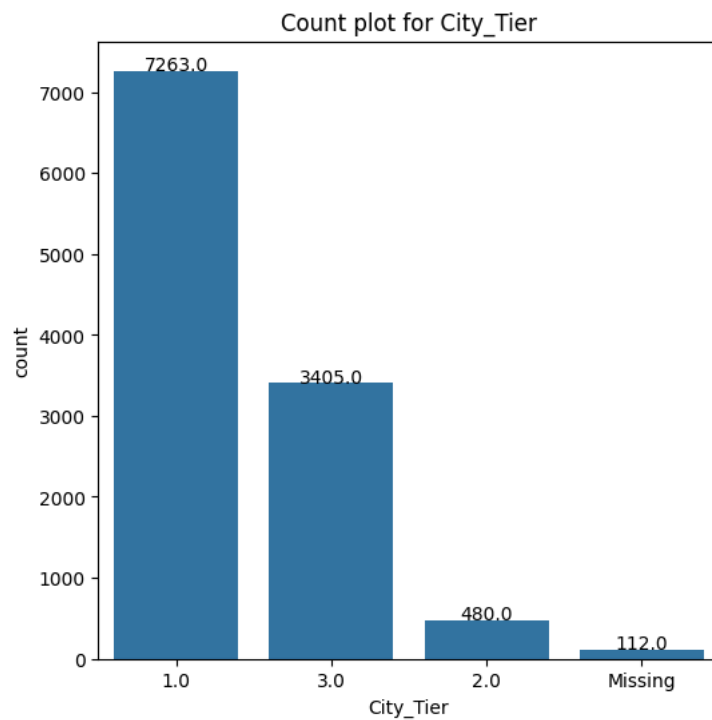
Other categorical variables

```
df = pd.DataFrame()
for i in cat_cols:
    plt.figure(figsize=(6,6))

    df[i] = CC[i].fillna("Missing")
    order = df[i].value_counts(ascending=False).index
    ax = sns.countplot(x=df[i],order=order)

    for p in ax.patches:
        height = p.get_height()
        ax.text(p.get_x()+p.get_width()/2., height+1, height, ha="center")

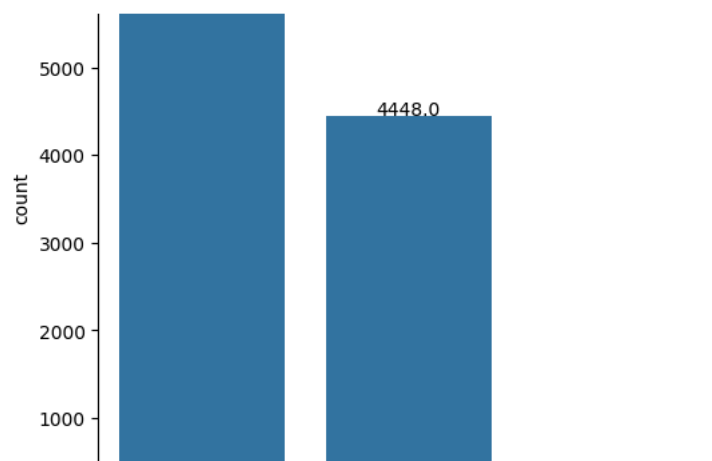
    plot_title = 'Count plot for ' + i
    plt.title(plot_title)
    plt.show();
```

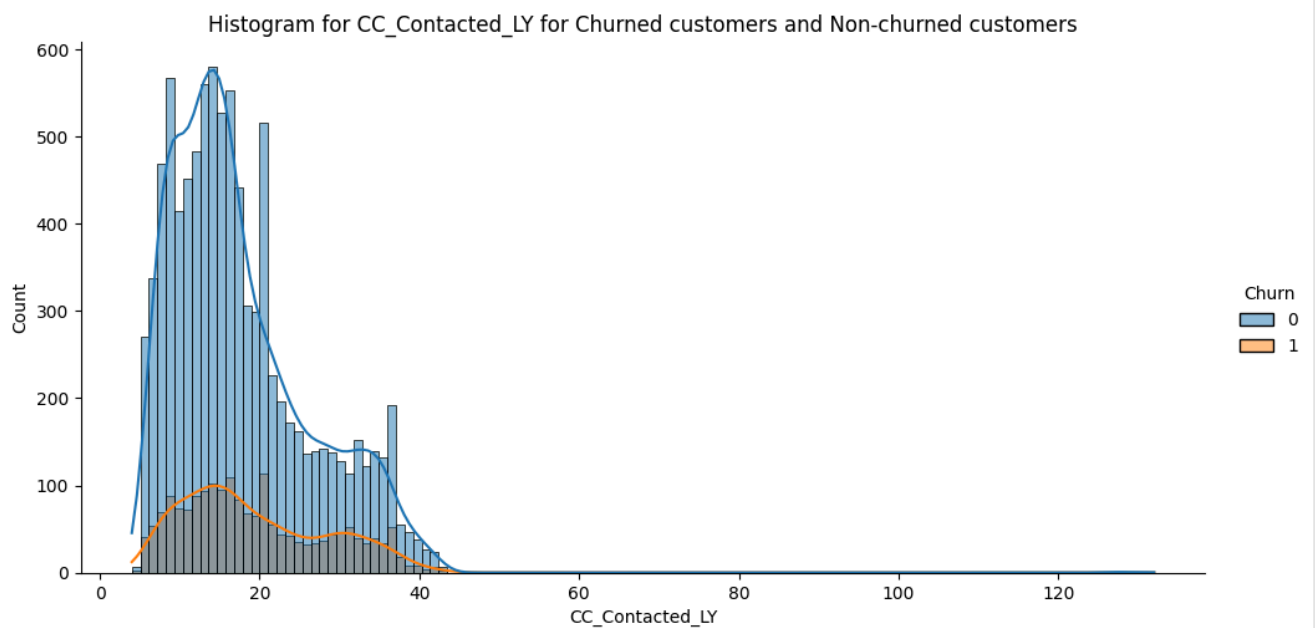
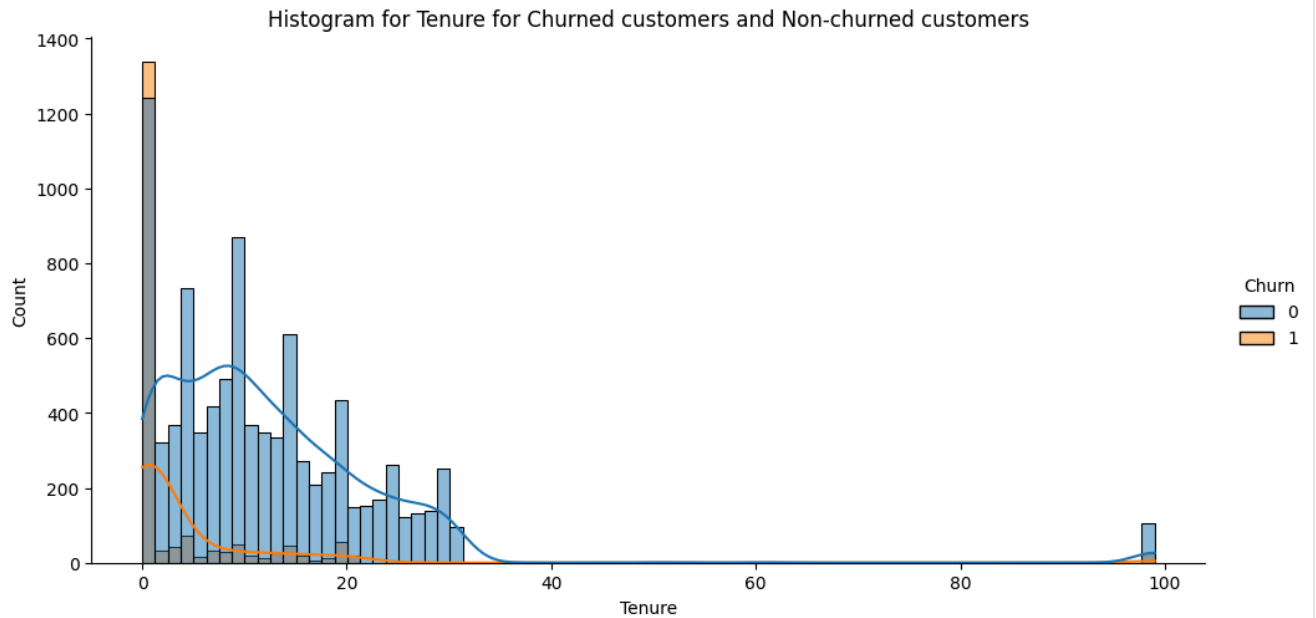


✓ Bivariate analysis - Numerical fields vs Churn

Histogram and boxplots with hue

```
for i in num_cols:
    sns.displot(x=CC[i], hue=CC['Churn'], kind='hist', kde=True, aspect=2)
    plot_title = "Histogram for " + i + " for Churned customers and Non-churned customers"
    plt.title(plot_title)
    plt.show();
```

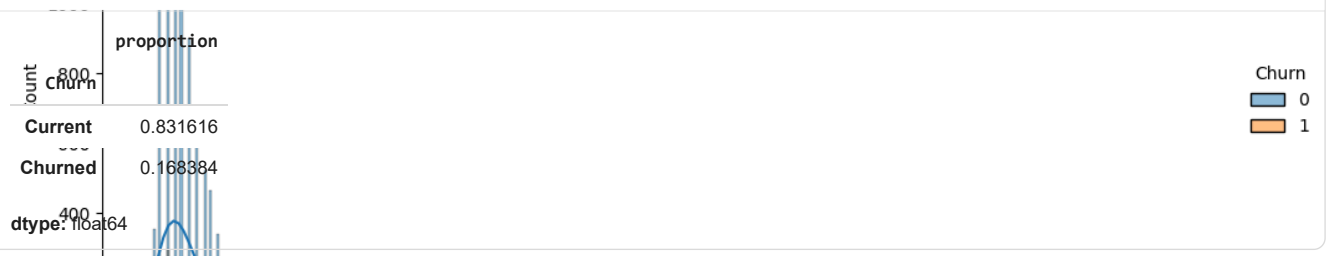




Histogram for rev_per_month for Churned customers and Non-churned customers

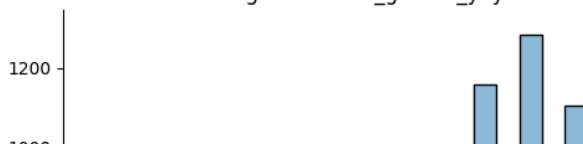
```
##Box plot errors if 0 and 1s are left as-is or datatype converted as object/categorical. Hence converting to string
##Need to convert back after analysis
CC['Churn'] = CC['Churn'].replace( {1: 'Churned', 0: 'Current'} )
```

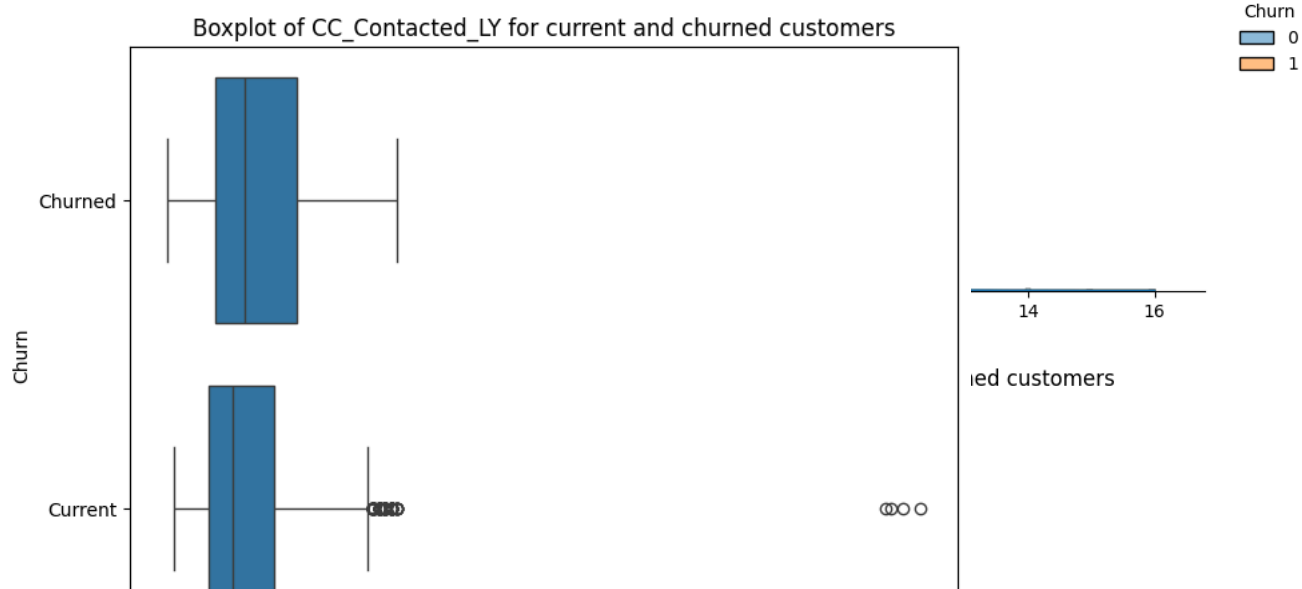
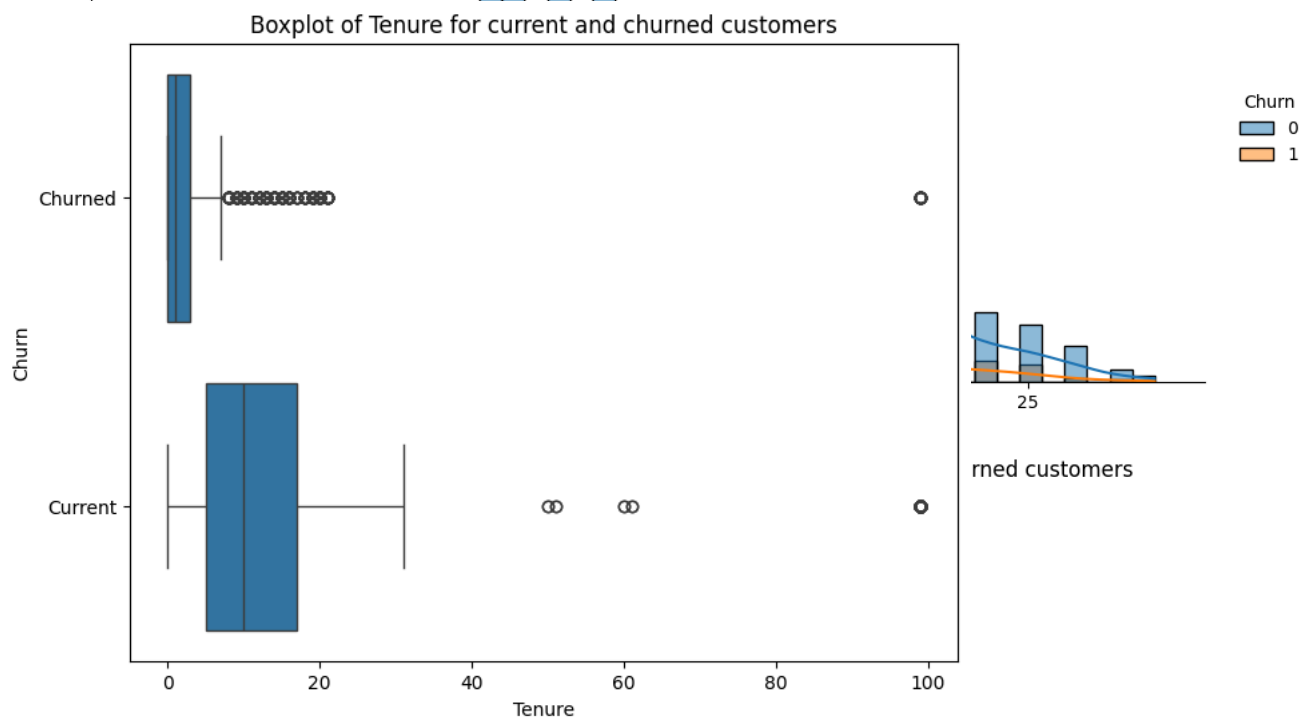
```
CC.Churn.value_counts(normalize=True)
```



```
for i in num_cols:
    plt.figure(figsize=(8,6))
    sns.boxplot(x=CC[i], y=CC['Churn'])
    plot_title = 'Boxplot of ' + i + " for current and churned customers"
    plt.title(plot_title)
    plt.show();
```

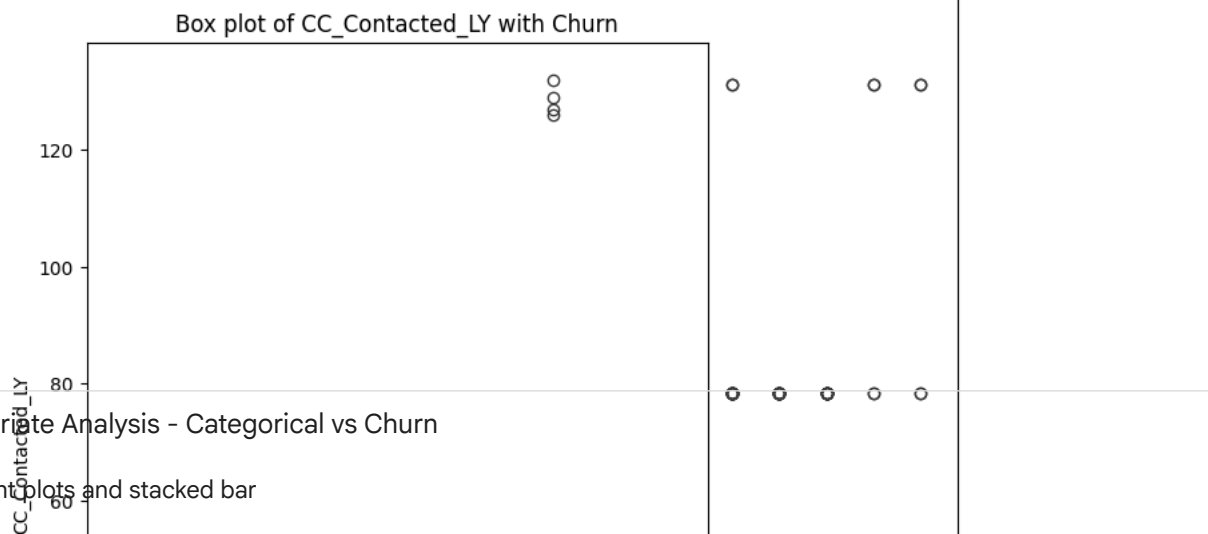
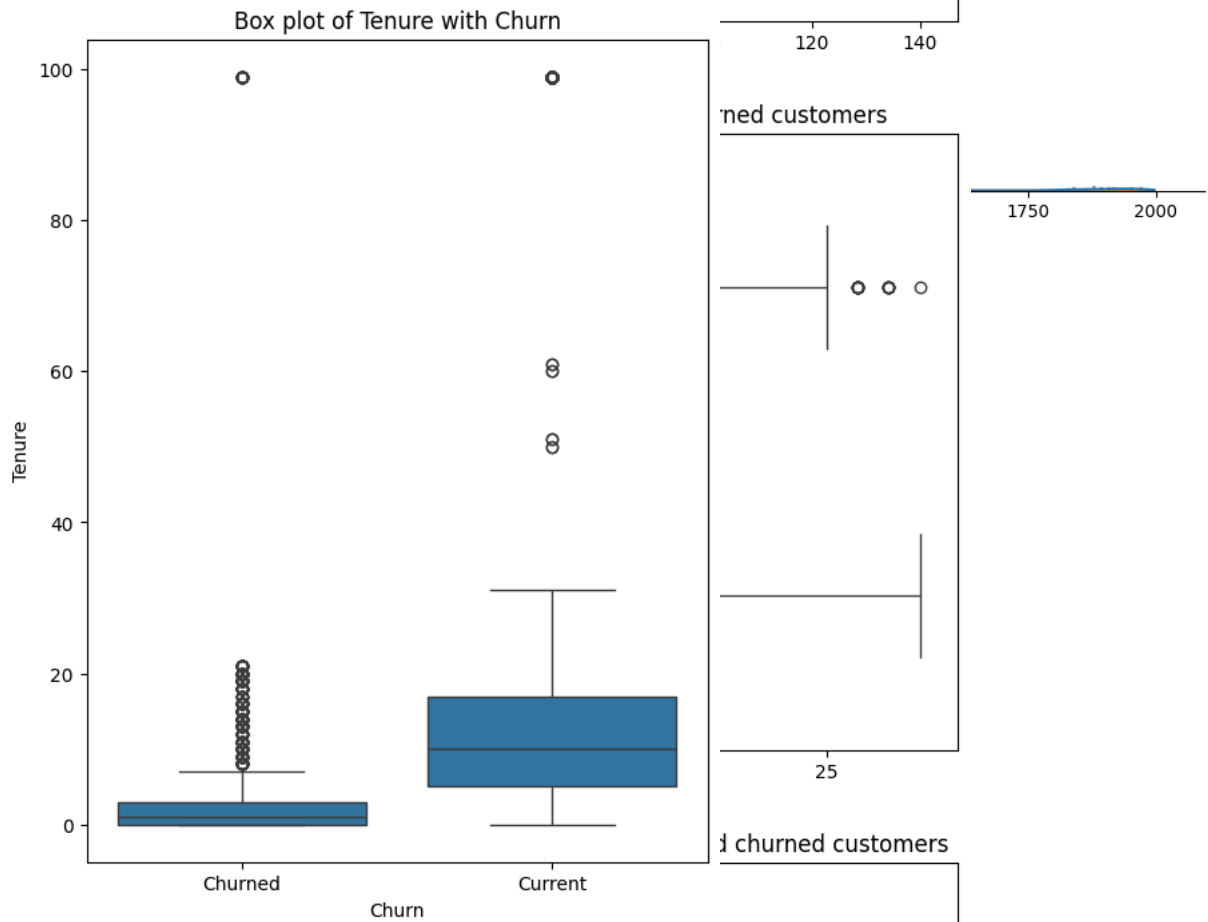
Histogram for rev_growth_yoy for Churned customers and Non-churned customers





```
for i in num_cols:
    plt.figure(figsize=(6,8))
    sns.boxplot(data = CC, y=i, x='Churn')
    plot_title = "Box plot of "+i+" with Churn"
    plt.title(plot_title)
    plt.show();
```



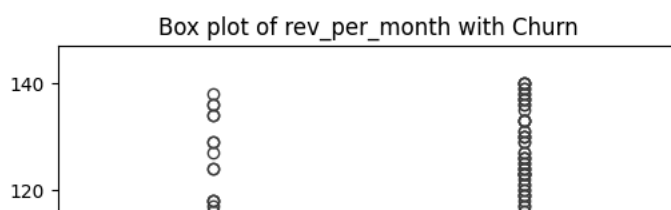
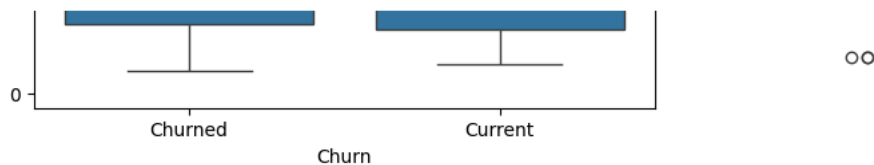


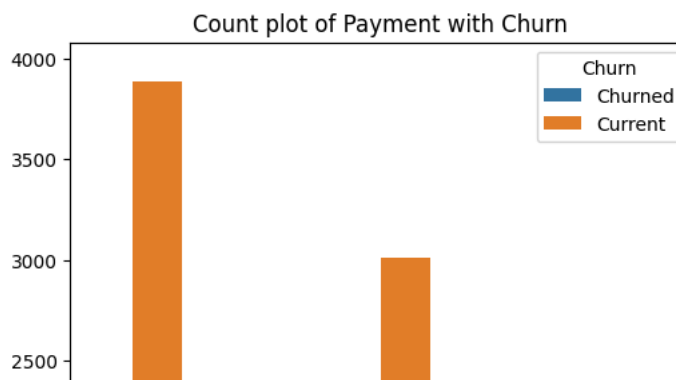
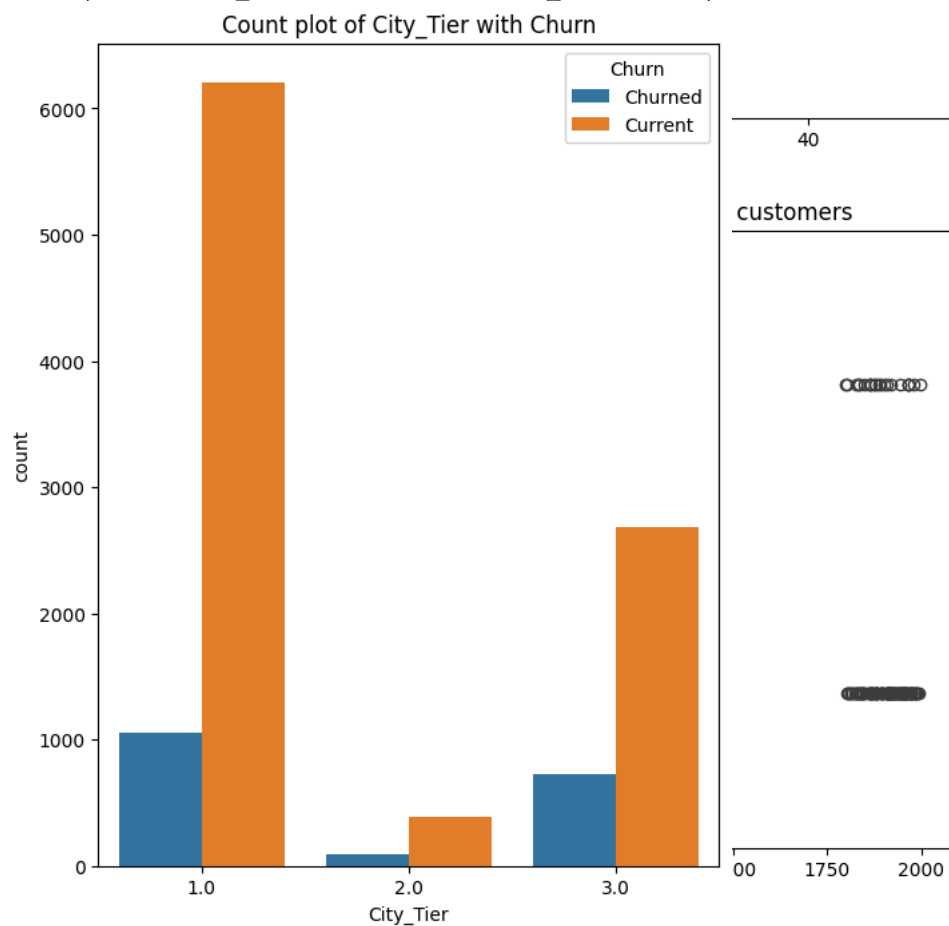
✓ Bivariate Analysis - Categorical vs Churn

Count plots and stacked bar

Countplot with hue

```
for i in cat_cols:
    plt.figure(figsize=(6,8))
    sns.countplot(data = CC, x=i, hue='Churn')
    plot_title = "Count plot of "+i+" with Churn"
    plt.title(plot_title)
    plt.show();
```

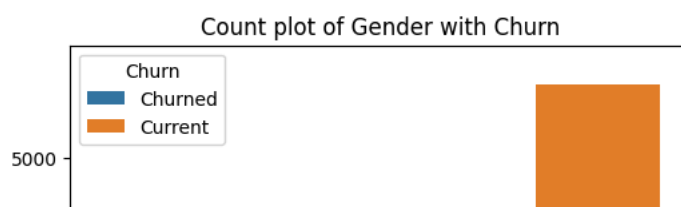




```
pd.pivot_table(CC, values='AccountID', index=['Churn'], columns=['Login_device'], aggfunc='count', margins=True)
```

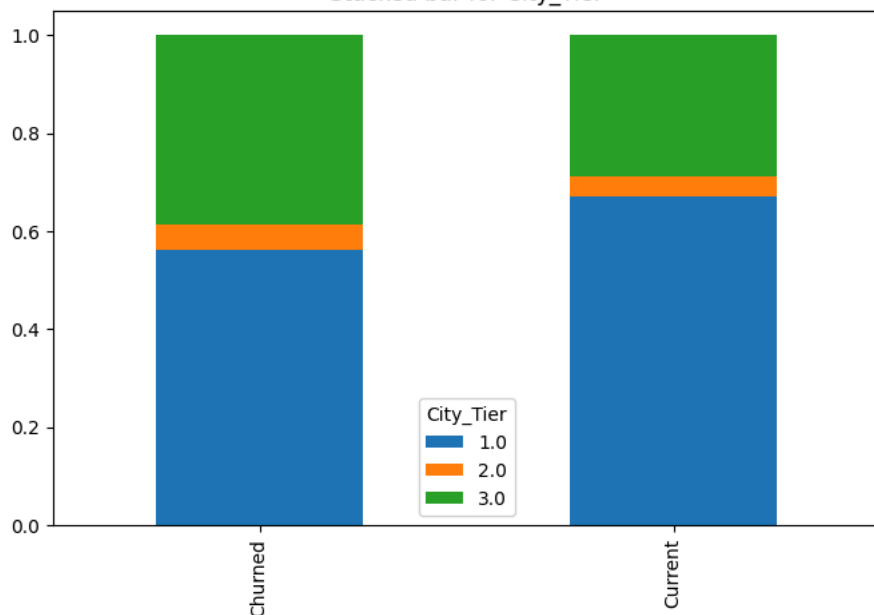
Churn	Login_device		
	Computer	Mobile	All
Churned	597	1172	1769
Current	2421	6310	8731
All	3018	7482	10500

```
for i in cat_cols:
    tab = pd.crosstab(CC['Churn'], CC[i], normalize='index')
    print(tab)
    #plt.figure(figsize=(12,6))
    tab.plot(kind='bar', stacked = True, figsize=(8,5))
    plot_title = "Stacked bar for " + i
    plt.title(plot_title)
    plt.show();
```



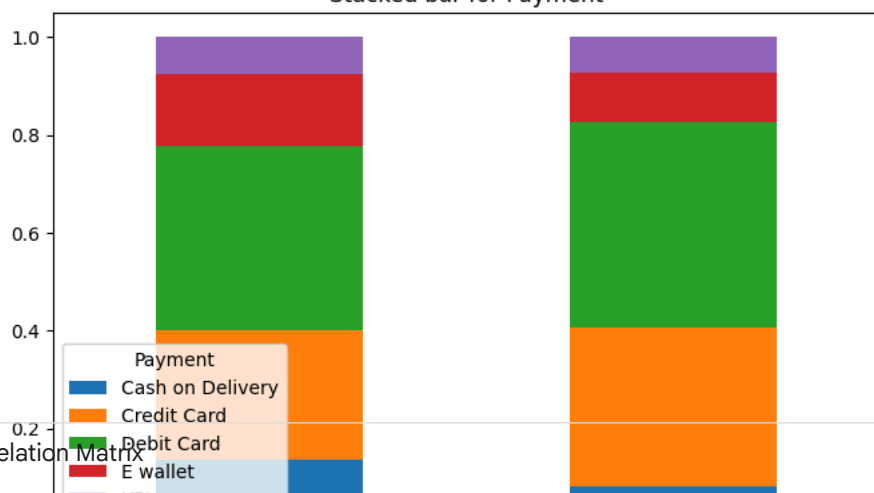
City_Tier	1.0	2.0	3.0
Churn			
Churned	0.562001	0.051091	0.386908
Current	0.669652	0.041428	0.288920

Stacked bar for City_Tier



Payment	Cash on Delivery	Credit Card	Debit Card	E wallet	UPI
Churn					
Churned	0.135539	0.266275	0.374600	0.147279	0.073192
Current	0.081923	0.324674	0.418778	0.101434	0.073192

Stacked bar for Payment



Correlation Matrix

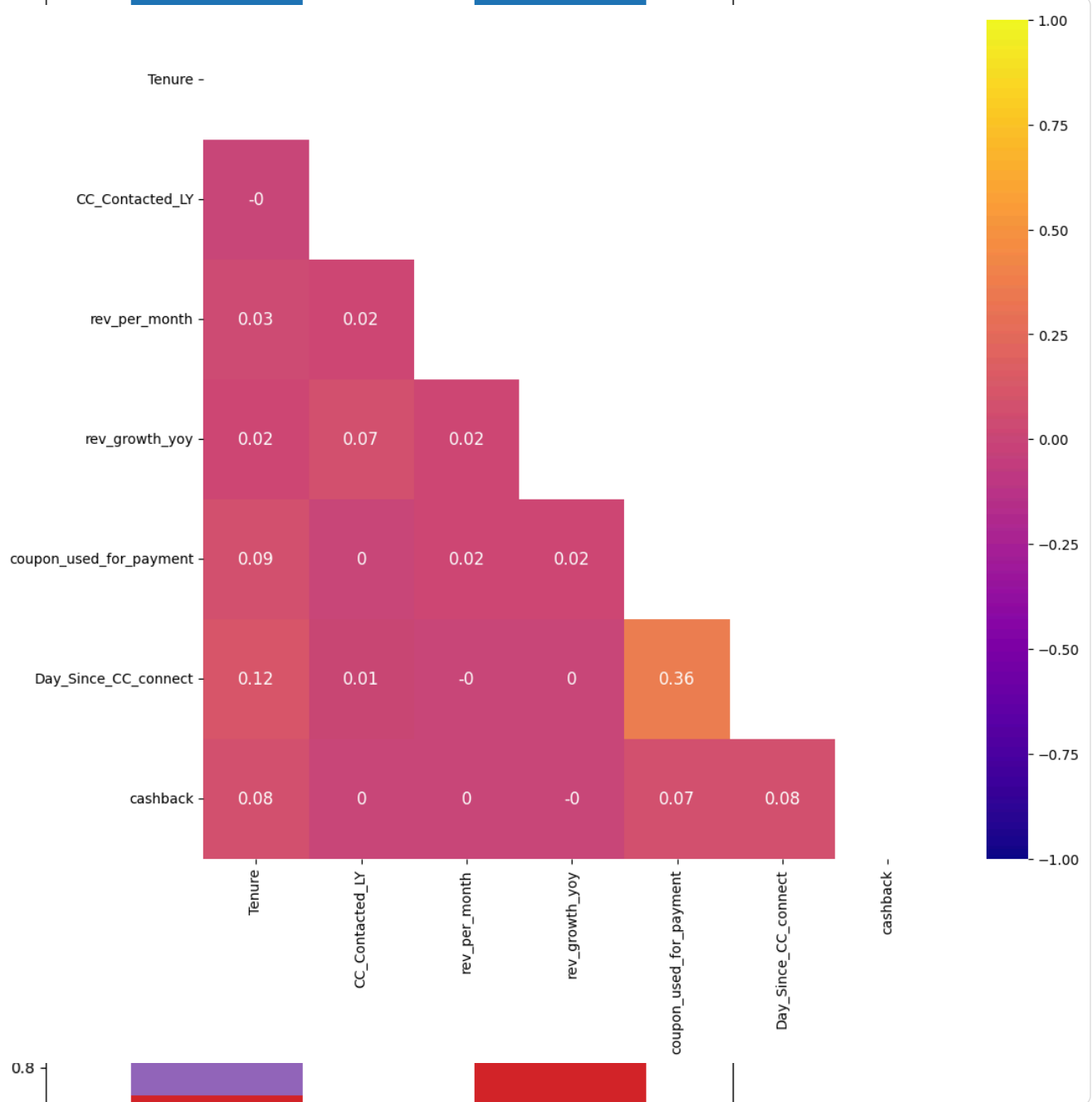
```
CC_num = CC[num_cols]
corr = CC_num.corr()
mask = np.triu(np.ones_like(corr, dtype = np.bool))

plt.figure(figsize = (12,11))
sns.heatmap(round(corr,2), annot=True, mask=mask, cmap='plasma', vmin=-1, vmax=1, annot_kws={'size':12});
```

	0.0	1.0	2.0	3.0	4.0	5.0
Churn	-					
Churned	0.367663	0.632337				
Current	0.405152	0.594848				

Stacked bar for Gender





Replacing 0s and 1s in Target column 'Churn'

```
CC['Churn'] = CC['Churn'].replace( {'Churned': 1, 'Current': 0} )
```

```
<ipython-input-80-16c730ac43b1>:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version of pandas. To suppress this warning, please call `pandas.api.types.is_integer_dtype` on the `other` parameter.
CC['Churn'] = CC['Churn'].replace( {'Churned': 1, 'Current': 0} )
```

```
CC['Churn'].dtypes
```

```
dtype('int64')
```

Removal of unwanted variables - Dropping AccountID

```
CC.drop(["AccountID", "rev_growth_yoy", "coupon_used_for_payment"], axis=1, inplace=True)
```

```
CC.shape
```

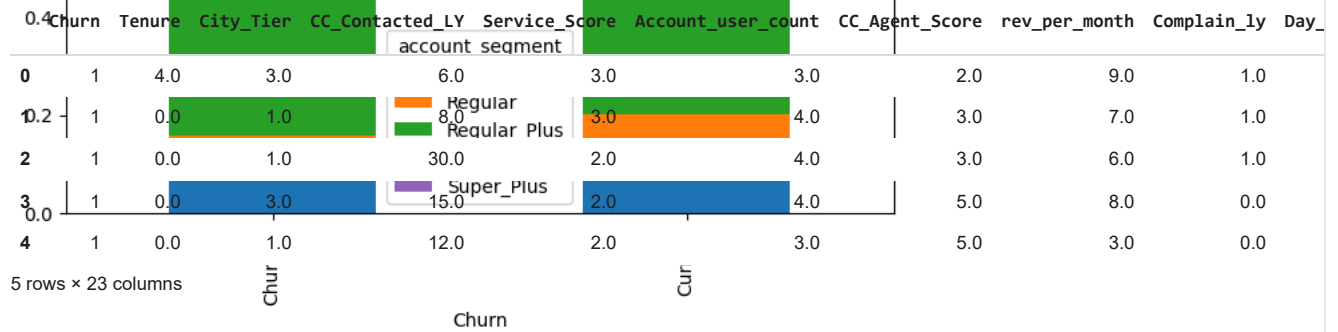
```
(11260, 16)
```

Stacked bar for account_segment

One hot/dummy encoding

```
OneHotCols = ['Payment', 'Gender', 'account_segment', 'Marital_Status', 'Login_device']
```

```
CC = pd.get_dummies(CC, columns=OneHotCols, drop_first=True) ### mass dummy encoding
CC.head()
```



```
CC.shape
Current (11260, 23)
```

Stacked bar for CC_Agent_Score

```
## Got to check if column names are ok after one hot encoding
CC.columns
```

```
Index(['Churn', 'Tenure', 'City_Tier', 'CC_Contacted_LY', 'Service_Score',
       'Account_user_count', 'CC_Agent_Score', 'rev_per_month', 'Complain_ly',
       'Day_Since_CC_connect', 'cashback', 'Payment_Credit Card',
       'Payment_Debit Card', 'Payment_E wallet', 'Payment_UPI', 'Gender_Male',
       'account_segment_Regular', 'account_segment_Regular_Plus',
       'account_segment_Super', 'account_segment_Super_Plus',
       'Marital_Status_Married', 'Marital_Status_Single',
       'Login_device_Mobile'],
      dtype='object')
```

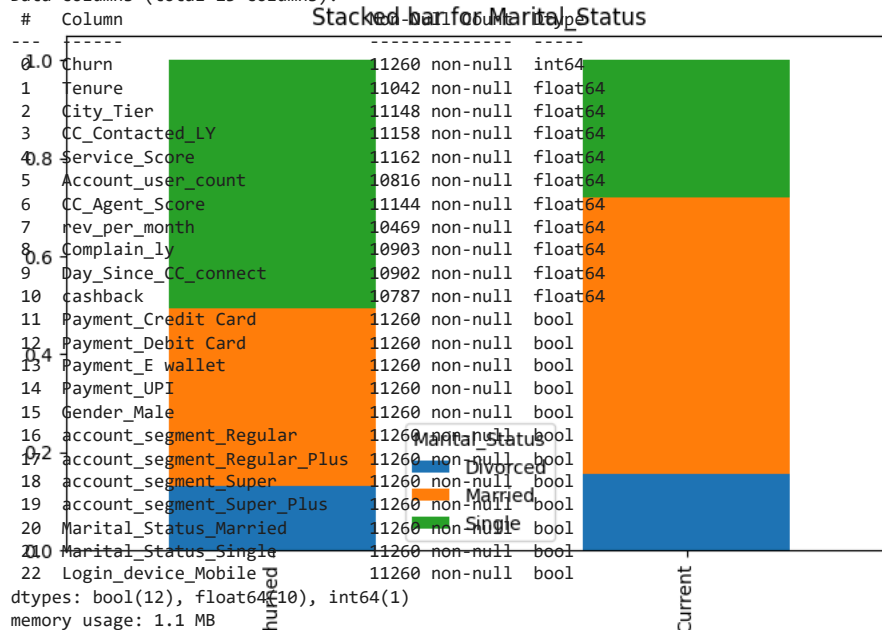
```
## Blanks in 'Credit card', 'E wallet' and 'Debit card' to be removed; Super Plus and Regular Plus also shortened
```

```
CC.columns = ['Churn', 'Tenure', 'City_Tier', 'CC_Contacted_LY', 'Service_Score',
              'User_Count', 'CC_Score', 'Rev_Permonth', 'Complain_LY',
              'Days_Since_CC', 'Cashback', 'Payment_Creditcard',
              'Payment_Debitcard', 'Payment_Ewallet', 'Payment_UPI', 'Gender_Male',
              'ACSegment_Regular', 'ACSegment_Regularplus',
              'ACSegment_Super', 'ACSegment_Superplus',
              'Maritalstatus_Married', 'Maritalstatus_Single',
              'Logindevice_Mobile']
```

```
CC.info()
```

```
Churn: 0.0 to 1.0, dtype: bool
Tenure: 0.0 to 11260, dtype: int64
City_Tier: 0.0 to 11260, dtype: float64
CC_Contacted_LY: 0.0 to 11260, dtype: float64
Service_Score: 0.0 to 11260, dtype: float64
Account_user_count: 0.0 to 11260, dtype: float64
CC_Agent_Score: 0.0 to 11260, dtype: float64
rev_per_month: 0.0 to 11260, dtype: float64
Complain_ly: 0.0 to 11260, dtype: float64
Day_Since_CC_connect: 0.0 to 11260, dtype: float64
cashback: 0.0 to 11260, dtype: float64
Payment_Credit Card: 0.0 to 11260, dtype: bool
Payment_Debit Card: 0.0 to 11260, dtype: bool
Payment_E wallet: 0.0 to 11260, dtype: bool
Payment_UPI: 0.0 to 11260, dtype: bool
Gender_Male: 0.0 to 11260, dtype: bool
account_segment_Regular: 0.0 to 11260, dtype: bool
account_segment_Regular_Plus: 0.0 to 11260, dtype: bool
account_segment_Super: 0.0 to 11260, dtype: bool
account_segment_Super_Plus: 0.0 to 11260, dtype: bool
Marital_Status_Married: 0.0 to 11260, dtype: bool
Marital_Status_Single: 0.0 to 11260, dtype: bool
Login_device_Mobile: 0.0 to 11260, dtype: bool
```

```
Data columns (total 23 columns):
```



Handling Missing Value

Churned 0.463451 0.537541
Current 0.765803 0.234197

Complain_ly

Stacked bar for Complain_ly

```
X = CC.drop('Churn', axis=1)
y = CC['Churn']
```

X.shape, y.shape

(11260, 22), (11260,)

Scaling before KNN imputing

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X = pd.DataFrame(scaler.fit_transform(X), columns = X.columns)
```

X.describe().T

		count	mean	std	min	25%	50%	75%	max
	Tenure	11042.0	-2.831360e-17	1.000045	-0.856038	-0.700749	-0.157237	0.386275	6.830775
	City_Tier	11148.0	3.728628e-17	1.000045	-0.754697	-0.714697	-0.714697	1.471157	1.471157
	CC_Contacted_LY	11158.0	-9.169937e-17	1.000045	-1.566394	-0.775691	-0.210902	0.579802	12.892189
	Service_Score	11162.0	-1.425923e-16	1.000045	-4.000444	-1.243919	0.134344	0.134344	2.890870
	Account_user_count	10816.0	1.261318e-16	1.000046	-2.632502	-0.677332	0.300253	0.300253	2.255423
Churned	CC_Agent_Score	11144.0	-1.836291e-16	1.000045	-1.497773	-0.772983	-0.048194	0.676596	1.401386
Current	rev_per_month	10469.0	-5.429689e-18	1.000048	-0.450293	-0.282355	-0.114416	0.053522	11.221437
1.0	Complain_ly	10903.0	-7.168642e-18	1.000046	-0.631867	-0.631867	-0.631867	1.582612	1.582612
	Day_Since_CC_connect	10902.0	-4.692632e-17	1.000046	-1.253070	-0.712160	-0.441704	0.910573	11.458334
0.8	cashback	10787.0	1.185665e-16	1.000046	-1.098427	-0.274423	-0.173445	0.021123	10.079715
	Payment_Credit Card	11260.0	-8.392734e-17	1.000044	-0.673120	-0.673120	-0.673120	1.485619	1.485619
	Payment_Debit Card	11260.0	1.100363e-16	1.000044	-0.829094	-0.829094	-0.829094	1.206136	1.206136
	Payment_E wallet	11260.0	-2.524131e-17	1.000044	-0.348108	-0.348108	-0.348108	-0.348108	2.872675
	Payment_UPI	11260.0	6.846704e-17	1.000044	-0.280626	-0.280626	-0.280626	-0.280626	3.563467
	Gender_Male	11260.0	1.262065e-18	1.000044	-1.213040	-1.213040	0.824375	0.824375	0.824375
	account_segment_Regular	11260.0	5.805500e-17	1.000044	-0.220039	-0.220039	-0.220039	-0.220039	4.544650
	account_segment_Regular_Plus	11260.0	1.561806e-16	1.000044	-0.760207	-0.760207	-0.760207	1.315431	1.315431
	account_segment_Super	11260.0	-1.097978e-16	1.000044	-0.751215	-0.751215	-0.751215	1.331177	1.331177
	account_segment_Super_Plus	11260.0	2.555682e-17	1.000044	-0.279888	-0.279888	-0.279888	-0.279888	3.572853
0.0	Marital_Status_Married	11260.0	5.048261e-18	1.000044	-1.041722	-1.041722	0.959949	0.959949	0.959949
	Marital_Status_Single	11260.0	-1.041204e-17	1.000044	-0.674374	-0.674374	-0.674374	1.482857	1.482857
	Login_device_Mobile	11260.0	6.310326e-18	1.000044	-1.407271	-1.407271	0.710595	0.710595	0.710595

KNN imputing

```
from sklearn.impute import KNNImputer

imputer = KNNImputer()
X = pd.DataFrame(imputer.fit_transform(X), columns = X.columns)
```

X.isnull().sum()

	0
Tenure	0
City_Tier	0
CC_Contacted_LY	0
Service_Score	0
Account_user_count	0
CC_Agent_Score	0
rev_per_month	0
Complain_ly	0
Day_Since_CC_connect	0
cashback	0
Payment_Credit Card	0
Payment_Debit Card	0
Payment_E wallet	0
Payment_UPI	0
Gender_Male	0
account_segment_Regular	0
account_segment_Regular_Plus	0
account_segment_Super	0
account_segment_Super_Plus	0
Marital_Status_Married	0
Marital_Status_Single	0
Login_device_Mobile	0

dtype: int64

```
X_inverse_with_outlier = pd.DataFrame(scaler.inverse_transform(X), columns=X.columns)
```

```
X_inverse_with_outlier.head()
```

	Tenure	City_Tier	CC_Contacted_LY	Service_Score	Account_user_count	CC_Agent_Score	rev_per_month	Complain_ly	Day_Since_C
0	4.0	3.0	6.0	3.0	3.0	2.0	9.0	1.0	
1	0.0	1.0	8.0	3.0	4.0	3.0	7.0	1.0	
2	0.0	1.0	30.0	2.0	4.0	3.0	6.0	1.0	
3	0.0	3.0	15.0	2.0	4.0	5.0	8.0	0.0	
4	0.0	1.0	12.0	2.0	3.0	5.0	3.0	0.0	

5 rows × 22 columns

```
## Writing the null treated dataset separately for models that are outlier resistant
```

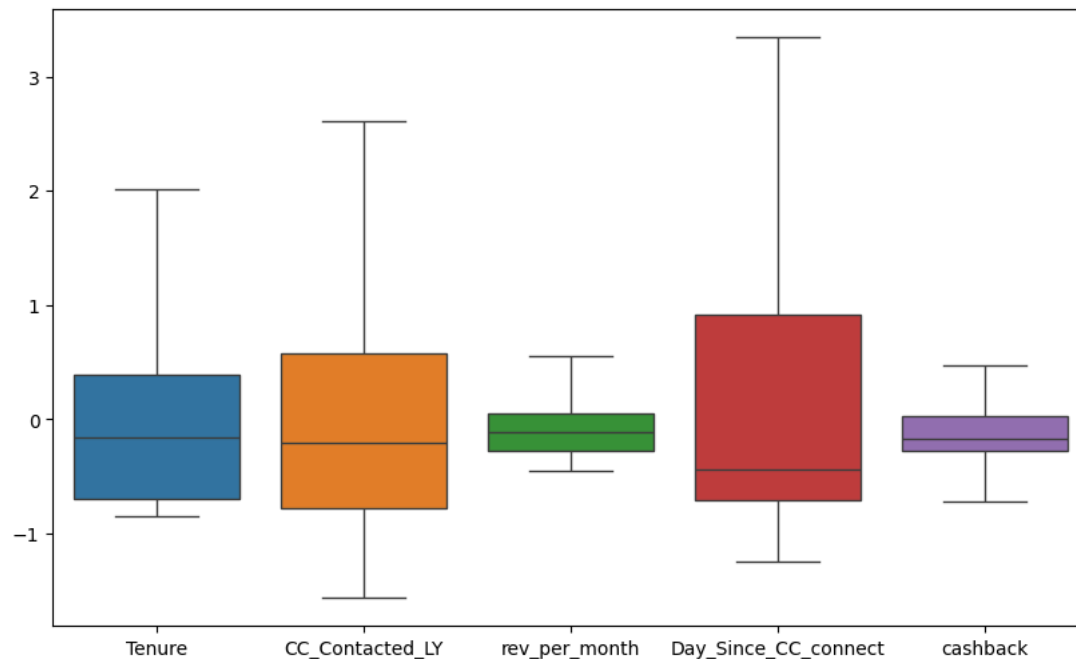
```
CC_with_outlier = pd.concat([round(X_inverse_with_outlier,0),y], axis=1)
CC_with_outlier.to_csv("CC_with_outlier.csv")
```

Outlier treatment

```
Outlier_treat_vars = ['Tenure', 'CC_Contacted_LY', 'rev_per_month', 'Day_Since_CC_connect', 'cashback']
```

```
Outlier_treat_vars = ['Tenure', 'CC_Contacted_LY', 'rev_per_month', 'Day_Since_CC_connect', 'cashback']
plt.figure(figsize=(10, 6))
sns.boxplot(data=X[Outlier_treat_vars])
```

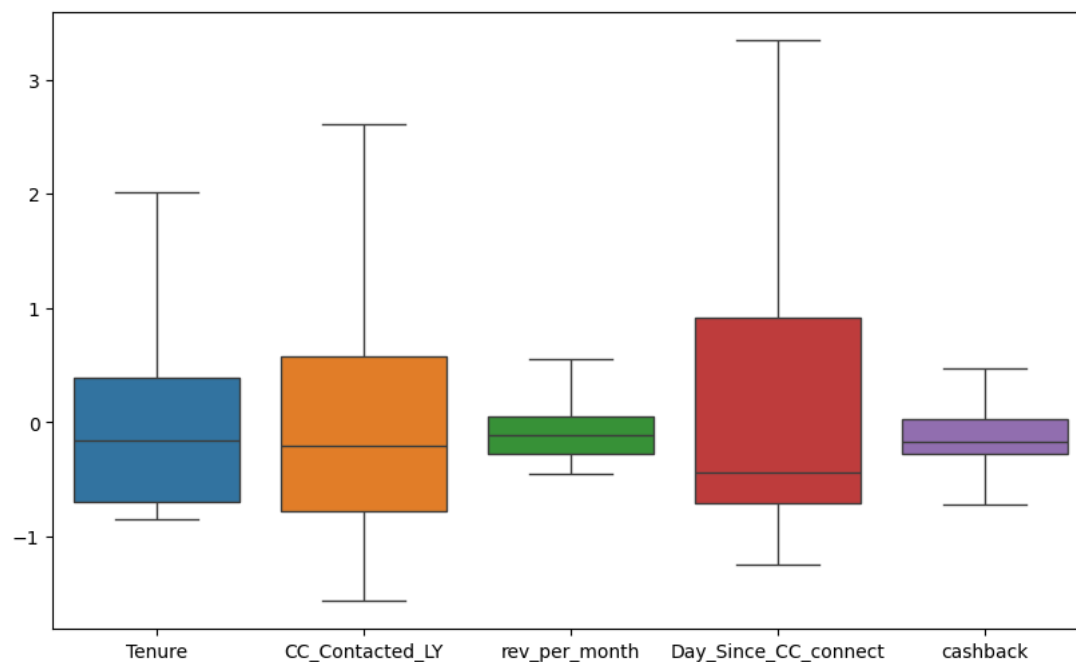
<Axes: >



```
def out_limit(col):
    Q1,Q3=col.quantile([0.25,0.75])
    IQR=Q3-Q1
    lower_range= Q1-(1.5 * IQR)
    upper_range= Q3+(1.5 * IQR)
    return lower_range, upper_range
```

```
for column in Outlier_treat_vars:
    lr,ur = out_limit(X[column])
    X[column] = np.where(X[column] > ur, ur, X[column])
    X[column] = np.where(X[column] < lr, lr, X[column])
```

```
plt.figure(figsize=(10,6))
sns.boxplot(data = X[Outlier_treat_vars]);
```



```
num_cols = ['Tenure', 'CC_Contacted_LY', 'rev_per_month', 'Day_Since_CC_connect', 'cashback'] # Updated num_cols
cols = ['Skewness']
pd.DataFrame(X[num_cols].skew().round(2), columns=cols)
```

	Skewness
Tenure	0.80
CC_Contacted_LY	0.80
rev_per_month	0.78
Day_Since_CC_connect	0.82
cashback	0.93

```
X_inverse_without_outlier = pd.DataFrame(scaler.inverse_transform(X), columns=X.columns)
```

```
X_inverse_without_outlier.columns
```

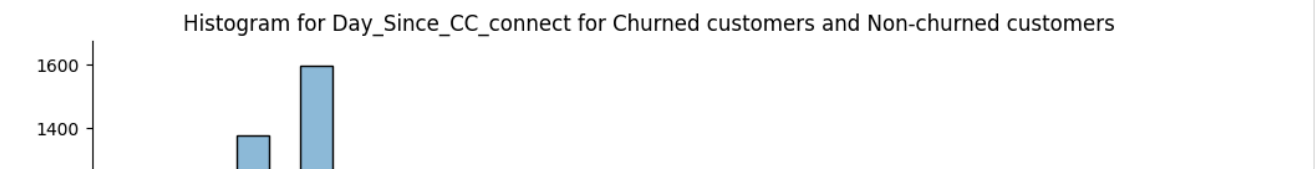
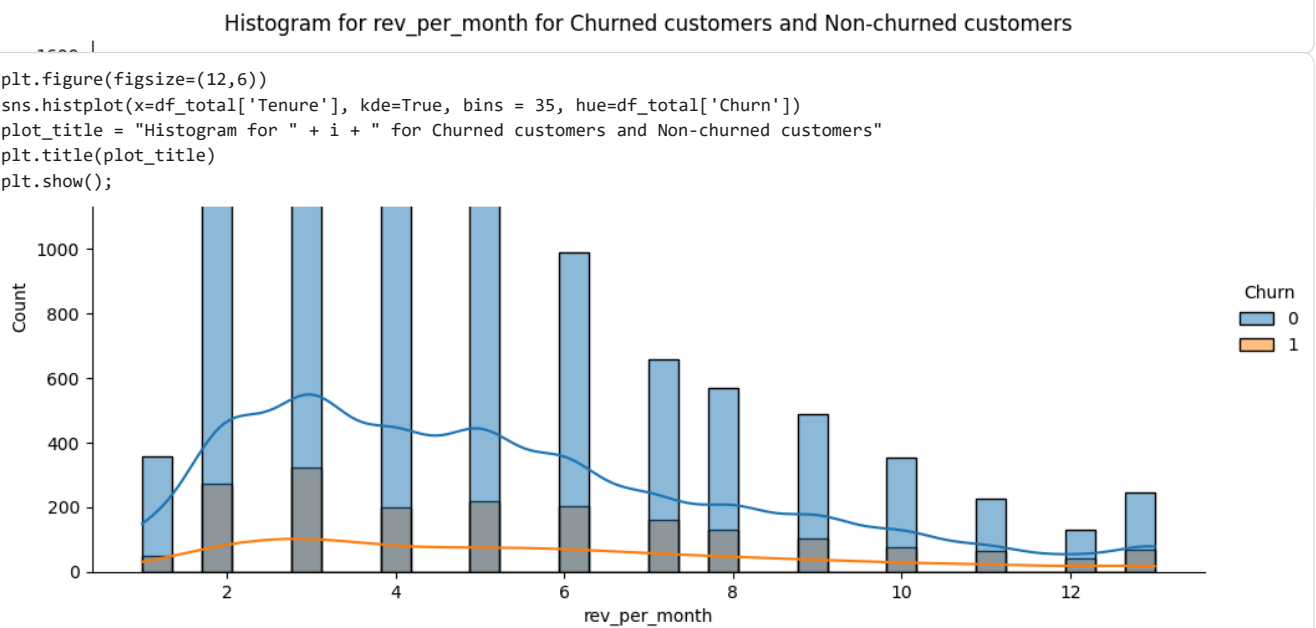
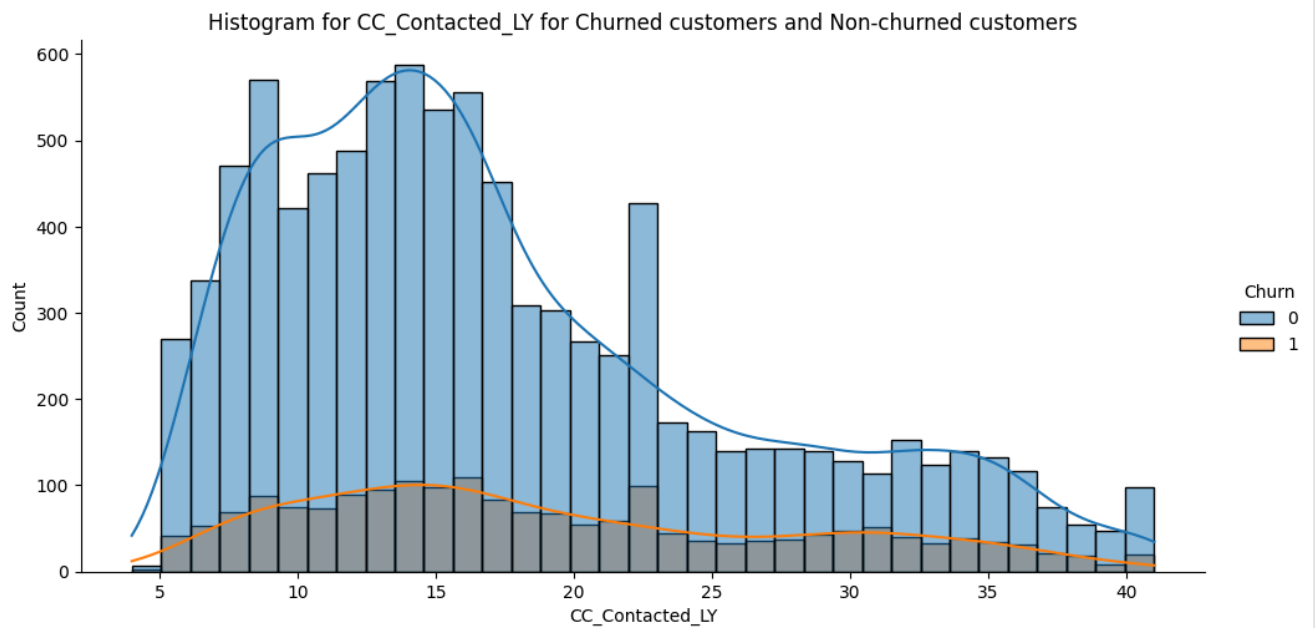
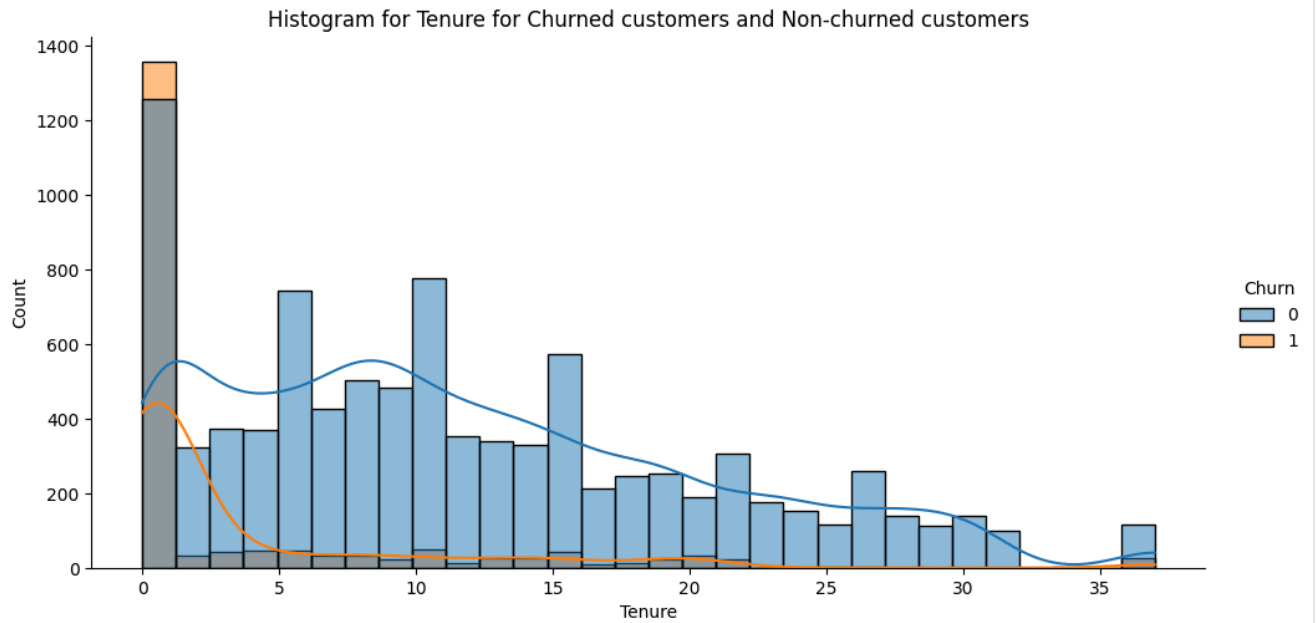
```
Index(['Tenure', 'City_Tier', 'CC_Contacted_LY', 'Service_Score',
      'Account_user_count', 'CC_Agent_Score', 'rev_per_month', 'Complain_ly',
      'Day_Since_CC_connect', 'cashback', 'Payment_Credit Card',
      'Payment_Debit Card', 'Payment_E wallet', 'Payment_UPI', 'Gender_Male',
      'account_segment_Regular', 'account_segment_Regular_Plus',
      'account_segment_Super', 'account_segment_Super_Plus',
      'Marital_Status_Married', 'Marital_Status_Single',
      'Login_device_Mobile'],
      dtype='object')
```

```
df_total = pd.concat([round(X_inverse_without_outlier,0),y],axis=1)
```

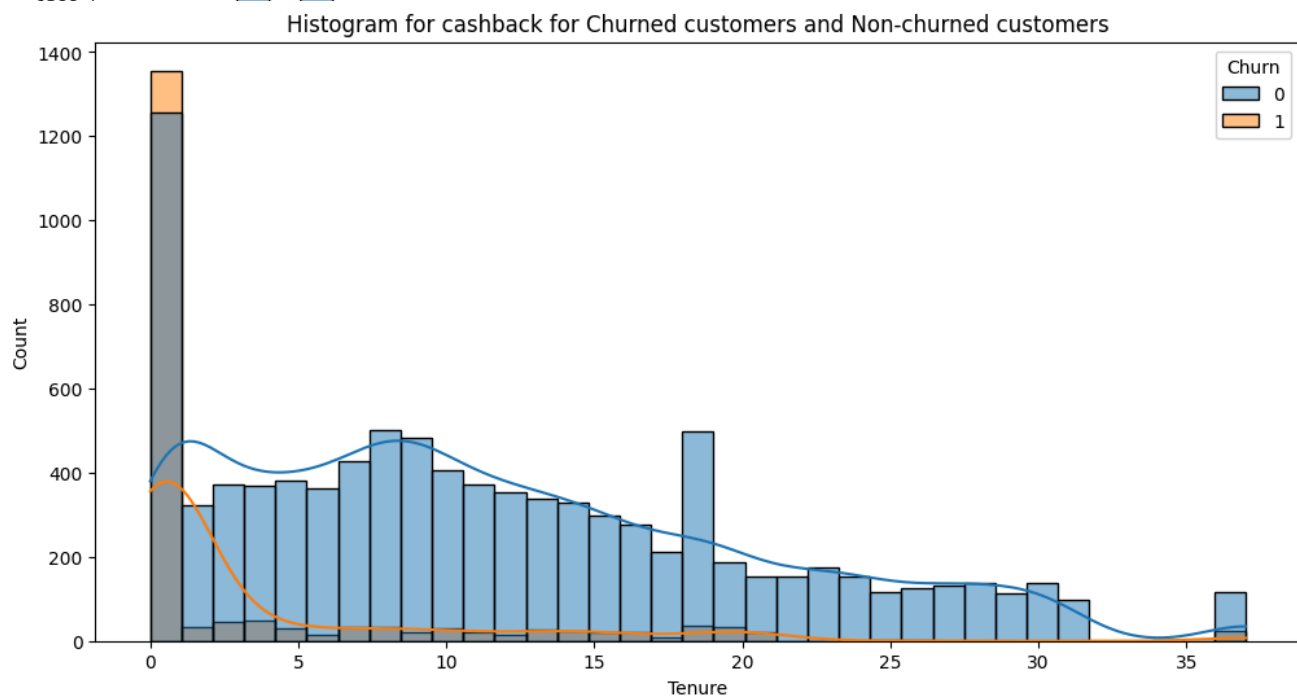
```
df_total.to_csv('CC_without_outlier.csv', index=False)
```

✓ Bivariate graphs after outlier & missing value treatment

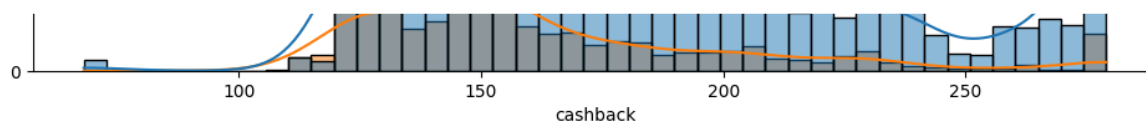
```
for i in num_cols:
    sns.displot(x=df_total[i], hue=df_total['Churn'], kind='hist', kde=True, aspect=2)
    plot_title = "Histogram for " + i + " for Churned customers and Non-churned customers"
    plt.title(plot_title)
    plt.show();
```

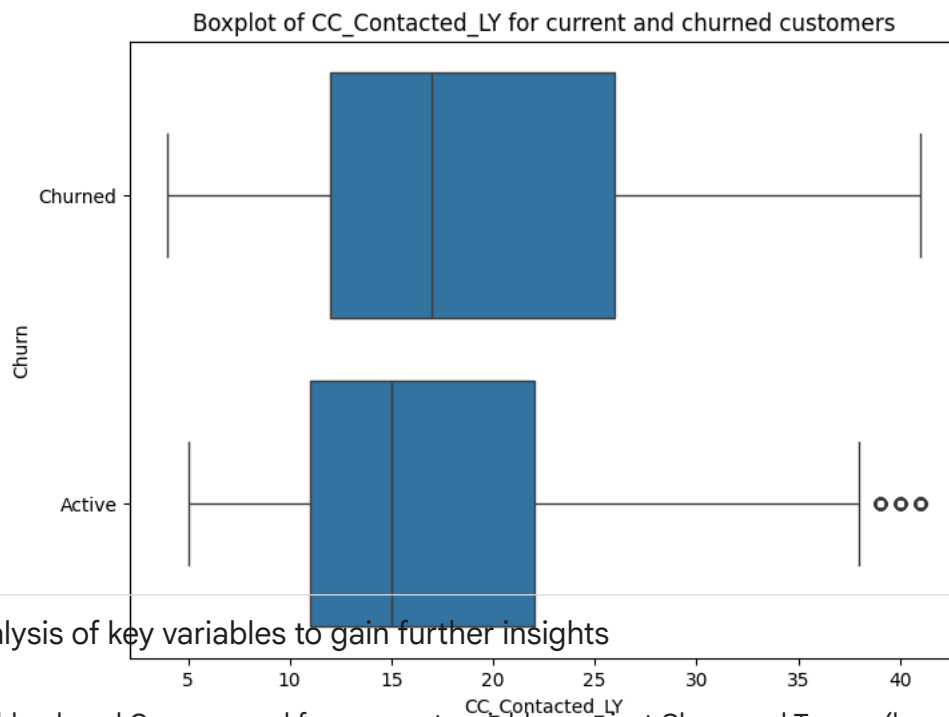
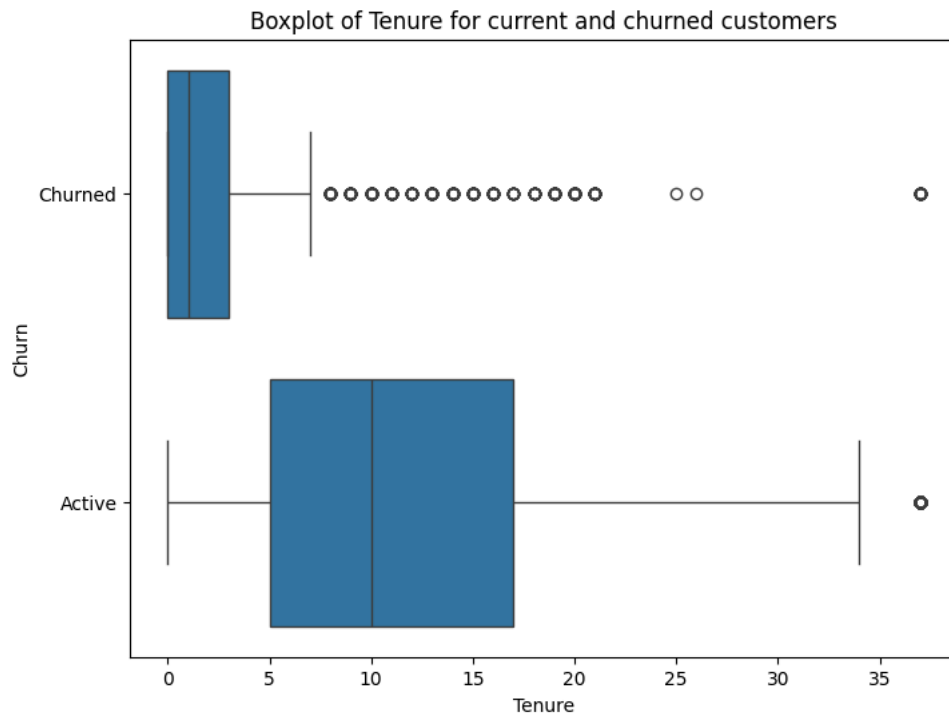


```
plt.figure(figsize=(12,6))
sns.histplot(x=df_total['Tenure'], kde=True, bins = 35, hue=df_total['Churn'])
plot_title = "Histogram for " + i + " for Churned customers and Non-churned customers"
plt.title(plot_title)
plt.show();
```



```
df_total_churn = df_total['Churn'].replace( {1: 'Churned', 0: 'Active'} ) ## y needs to be categorical
for i in num_cols:
    plt.figure(figsize=(8,6))
    sns.boxplot(x=df_total[i], y=df_total_churn)
    plot_title = 'Boxplot of ' + i + " for current and churned customers"
    plt.title(plot_title)
    plt.show();
```



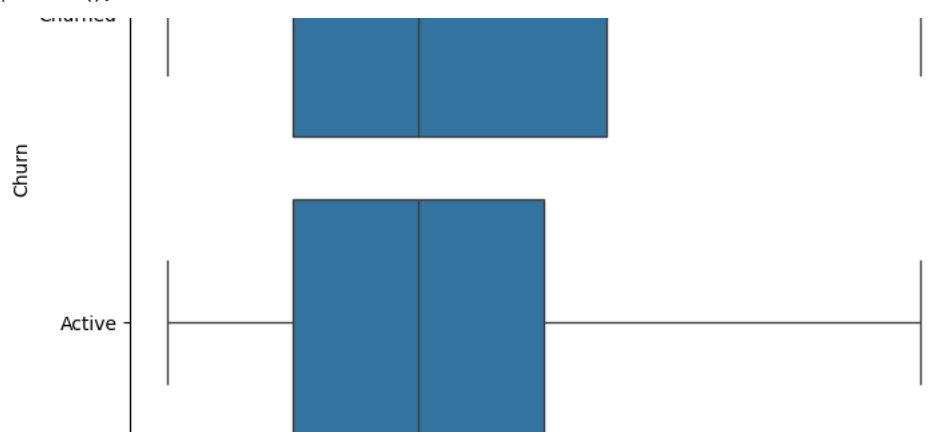


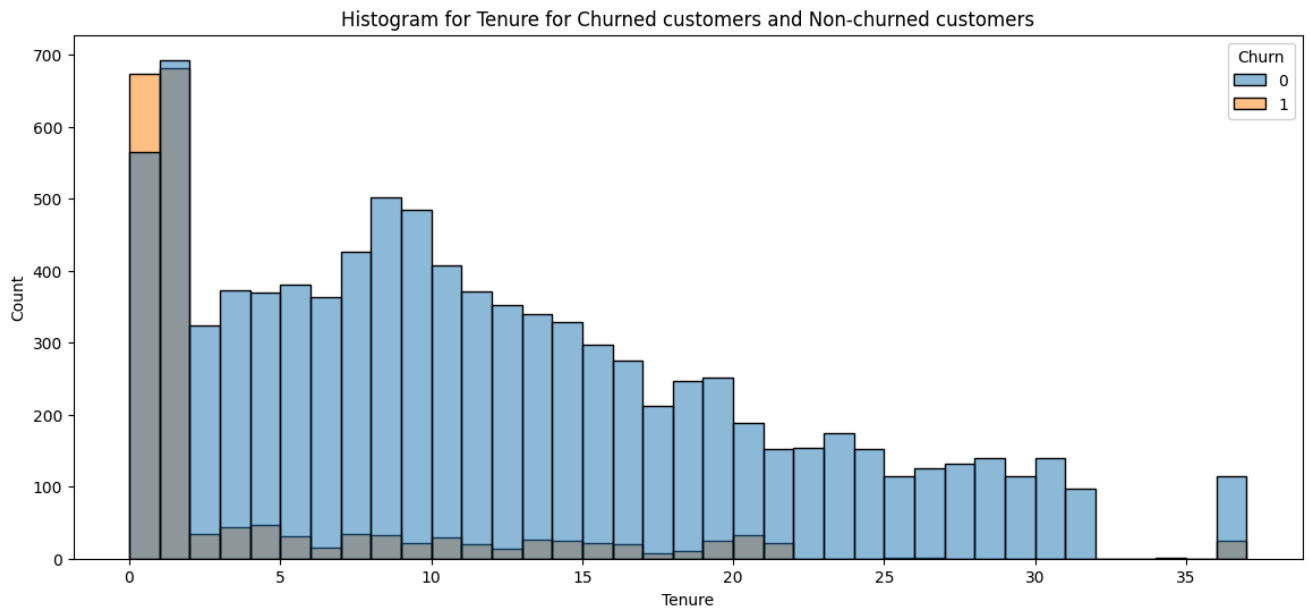
✓ Analysis of key variables to gain further insights

✓ Cashback and Coupon used for payment variables against Churn and Tenure (low and high bins)

Boxplot of rev_per_month for current and churned customers

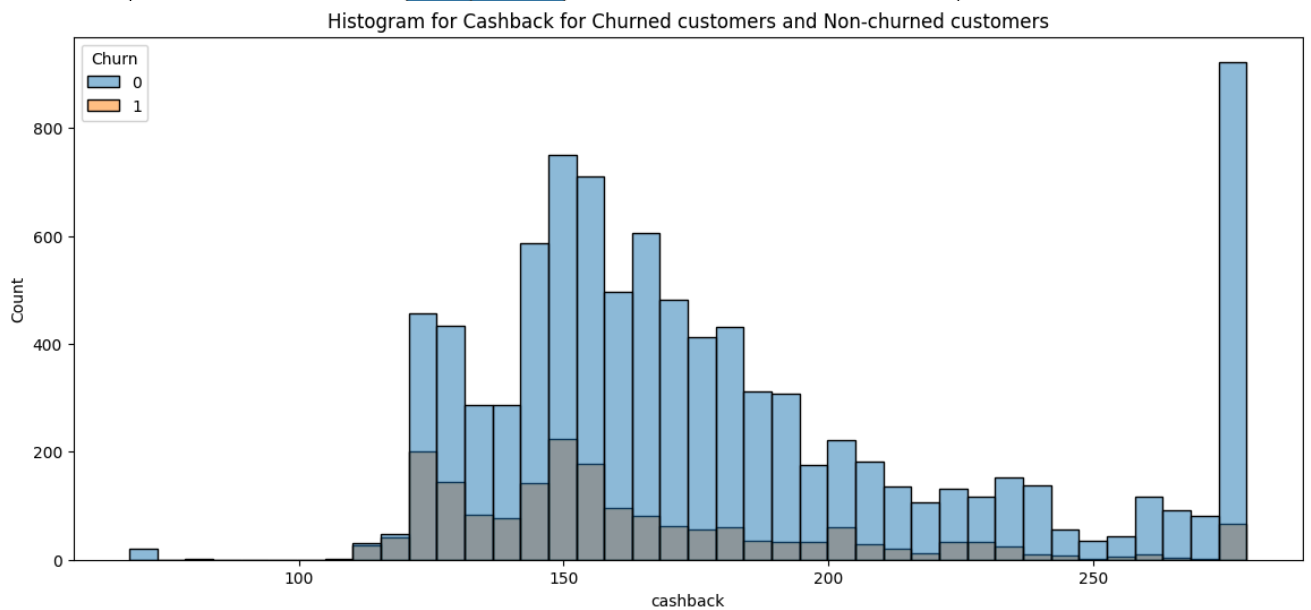
```
plt.figure(figsize=(14,6))
sns.histplot(x=df_total['Tenure'], bins = 37, hue=df_total['Churn'])
plot_title = "Histogram for " + "Tenure" + " for Churned customers and Non-churned customers"
plt.title(plot_title)
plt.show();
```





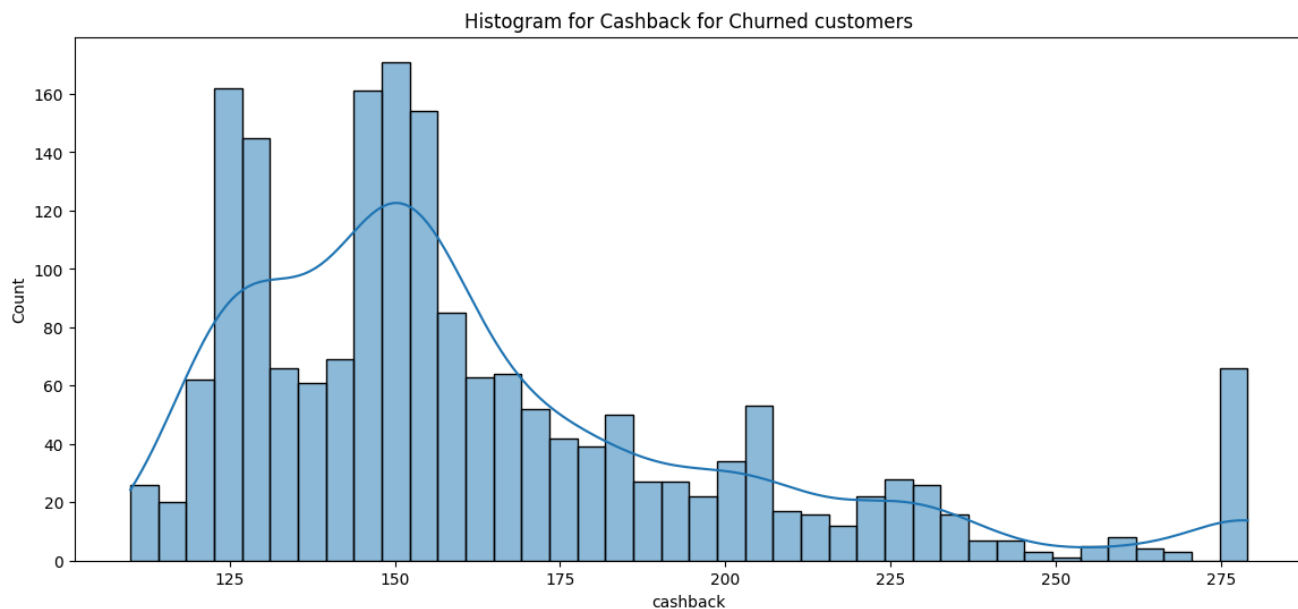
```
### Cashback histplot wrt churn
```

```
plt.figure(figsize=(14,6))
sns.histplot(x=df_total['cashback'], bins = 40, hue=df_total['Churn'])
plot_title = "Histogram for " + "Cashback" + " for Churned customers and Non-churned customers"
plt.title(plot_title)
plt.show();
```



```
### Cashback offers for only churned customers
```

```
plt.figure(figsize=(14,6))
sns.histplot(x=df_total[df_total['Churn']==1]['cashback'], kde=True, bins = 40)
plot_title = "Histogram for " + "Cashback" + " for Churned customers"
plt.title(plot_title)
plt.show();
```



```
### MEDIAN CASHBACK FOR CHURNED AND NON-CHURNED CUSTOMERS
```

```
df_total[df_total['Churn']==1]['cashback'].median(), df_total[df_total['Churn']==0]['cashback'].median()
```

```
(153.0, 168.0)
```

```
### Statistical description of cashback for churned customers
```

```
pd.DataFrame(df_total[df_total['Churn']==1]['cashback'].describe() )
```

	cashback
count	1896.000000
mean	163.103376
std	38.422260
min	110.000000
25%	135.000000
50%	153.000000
75%	179.000000
max	279.000000

```
### Statistical description of cashback for active customers
```

```
pd.DataFrame(df_total[df_total['Churn']==0]['cashback'].describe() )
```

	cashback
count	9364.000000
mean	181.617044
std	46.632509
min	68.000000
25%	149.000000
50%	168.000000
75%	205.000000
max	279.000000

```
df_temp=df_total.copy()
```

```
df_temp['Tenure_bin'] = np.where(df_temp['Tenure'] <= 2 , 'Tenure<=2', 'Other_Tenure')
```

```
df_temp['Tenure_bin'].value_counts()
```

```

count
Tenure_bin
Other_Tenure    8288
Tenure<=2       2972

```

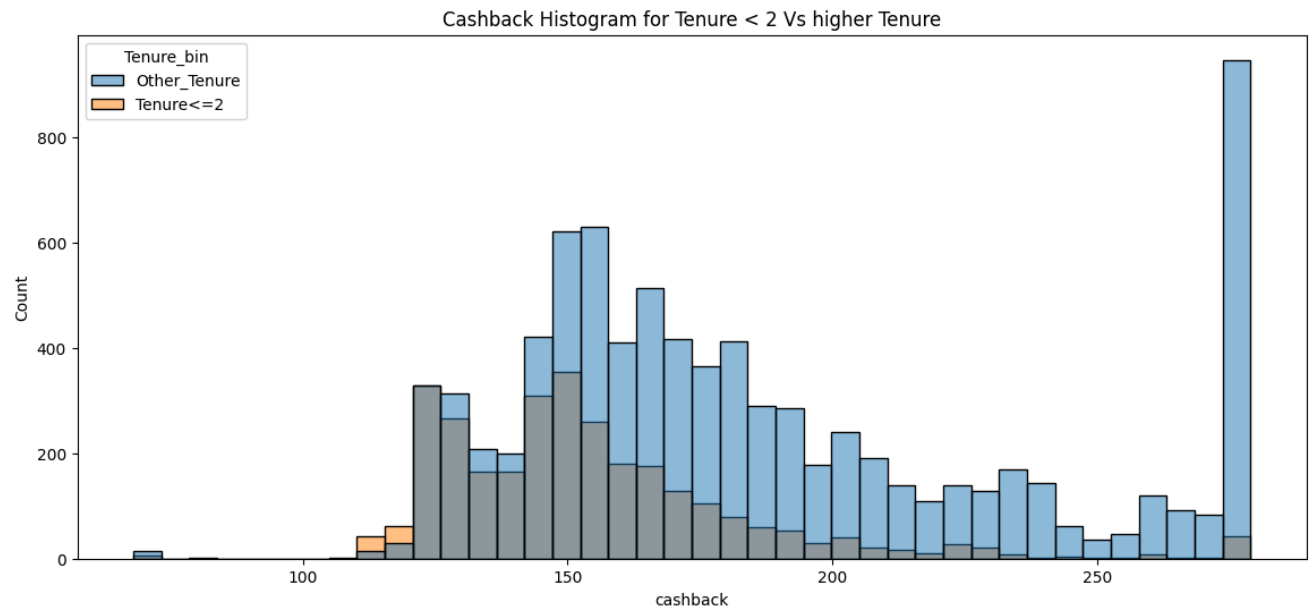
```
dtype: int64
```

```
## Histplot of cashback with Tenure as hue
```

```

plt.figure(figsize=(14,6))
sns.histplot(x=df_temp['cashback'], bins = 40, hue=df_temp['Tenure_bin'])
plot_title = "Cashback Histogram" + " for Tenure < 2 Vs higher Tenure"
plt.title(plot_title)
plt.show();

```

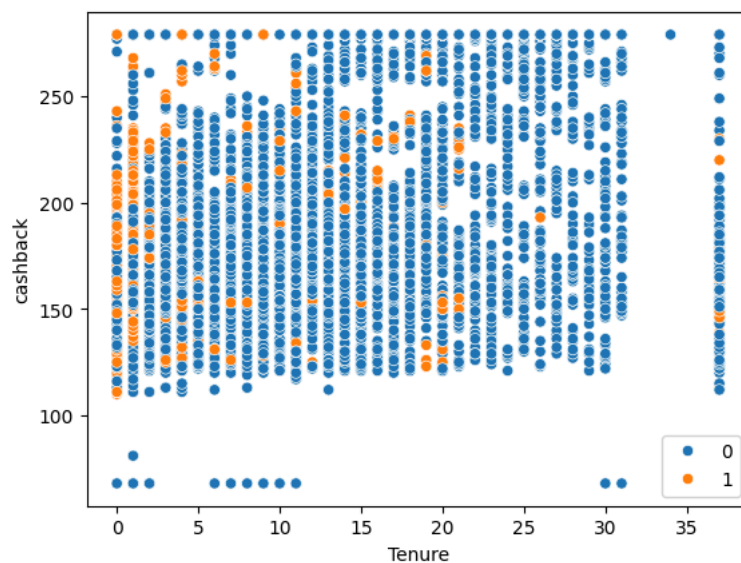


```
### Scatter plot of Tenure and Cashback
```

```

sns.scatterplot(x=df_total['Tenure'], y=df_total['cashback'], hue=df_total['Churn'])
plt.legend(loc='lower right');

```



```
CC_predictor.columns
```

```

Index(['Tenure', 'City_Tier', 'CC_Contacted_LY', 'Payment', 'Gender',
      'Service_Score', 'Account_user_count', 'account_segment',

```

```
'CC_Agent_Score', 'Marital_Status', 'rev_per_month', 'Complain_ly',  
'rev_growth_yoy', 'coupon_used_for_payment', 'Day_Since_CC_connect',  
'cashback', 'Login_device'],  
dtype='object')
```

```
CC_temp = CC_predictor.copy()  
CC_temp['Churn'] = df_total['Churn']
```

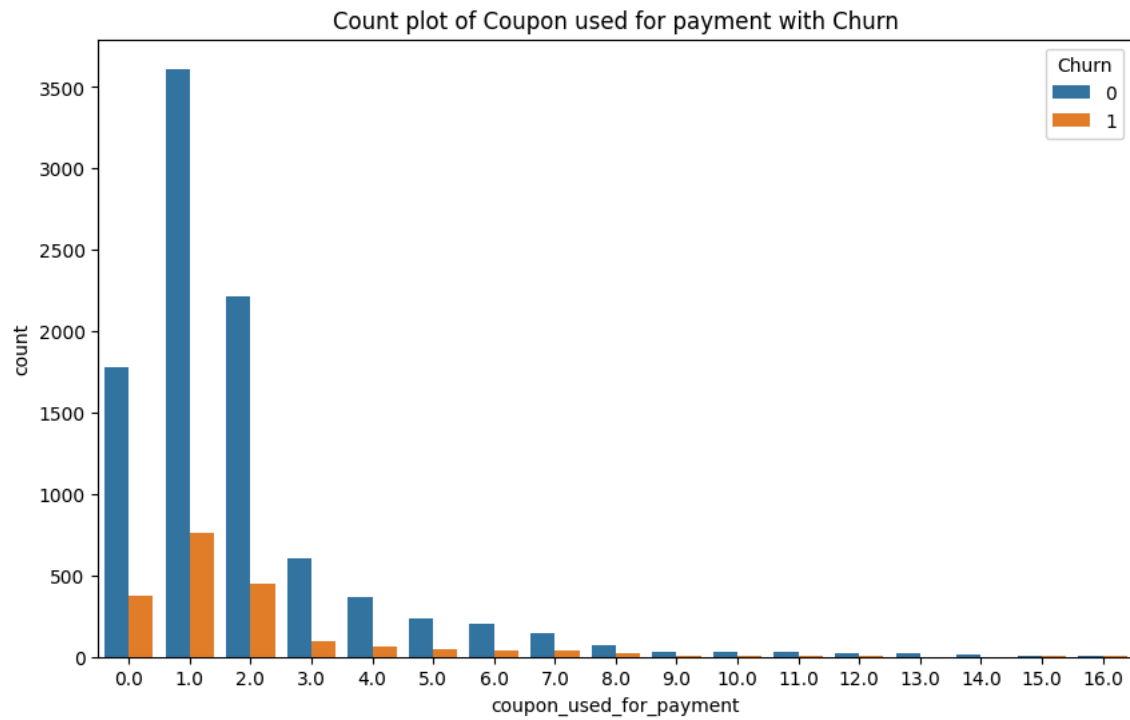
```
CC_temp.coupon_used_for_payment.value_counts()
```

	count
coupon_used_for_payment	
1.0	4373
2.0	2656
0.0	2150
3.0	698
4.0	424
5.0	284
6.0	234
7.0	184
8.0	88
9.0	34
10.0	34
11.0	30
12.0	26
13.0	22
14.0	12
15.0	4
16.0	4

dtype: int64

```
### Distribution of coupon_used_for_payment by churn
```

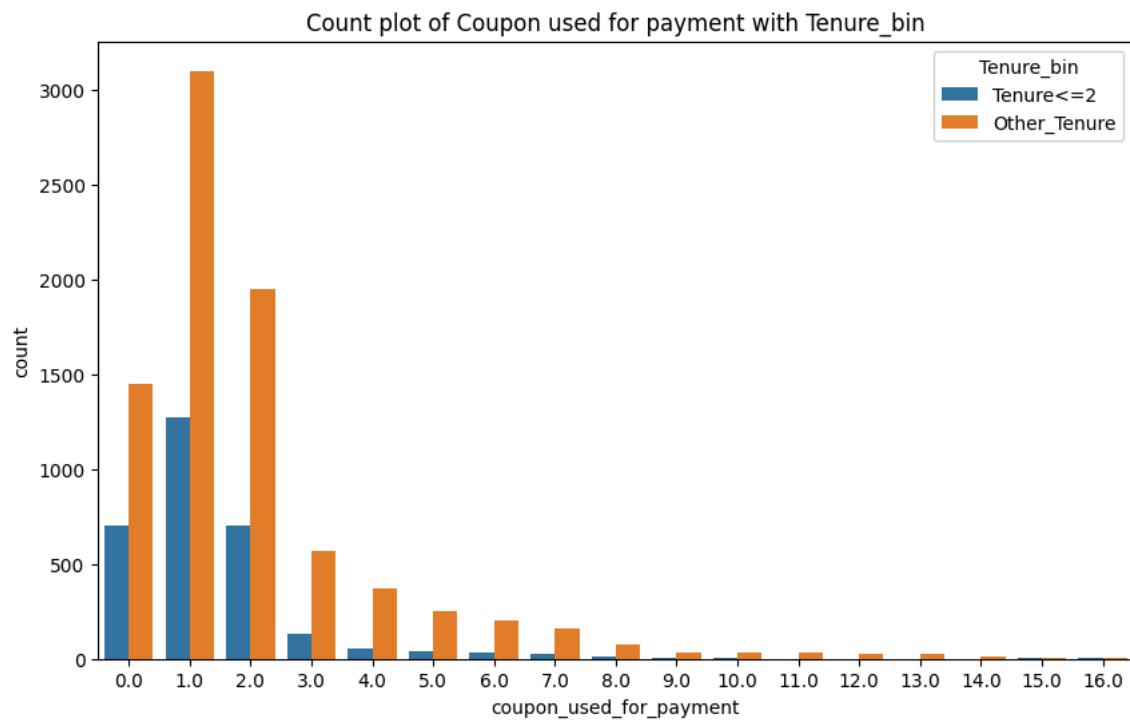
```
plt.figure(figsize=(10,6))  
sns.countplot(data = CC_temp, x='coupon_used_for_payment', hue='Churn')  
plot_title = "Count plot of "+"Coupon used for payment"+" with Churn"  
plt.title(plot_title)  
plt.show();
```



```
CC_temp['Tenure_bin'] = df_temp['Tenure_bin']
```

```
### Distribution of coupon_used_for_payment by tenure bin
```

```
plt.figure(figsize=(10,6))
sns.countplot(data = CC_temp, x='coupon_used_for_payment', hue='Tenure_bin')
plot_title = "Count plot of "+"Coupon used for payment"+" with Tenure_bin"
plt.title(plot_title)
plt.show();
```



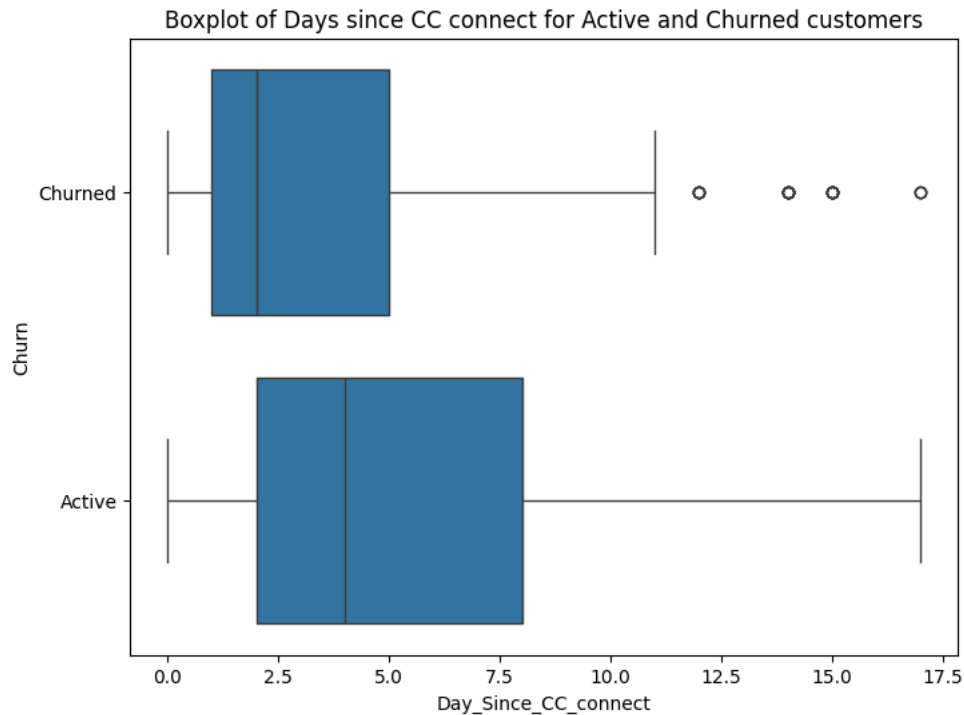
✓ Days since customer care contact, CC contacted last year, Complaint raised last year

```
CC_temp.columns
```

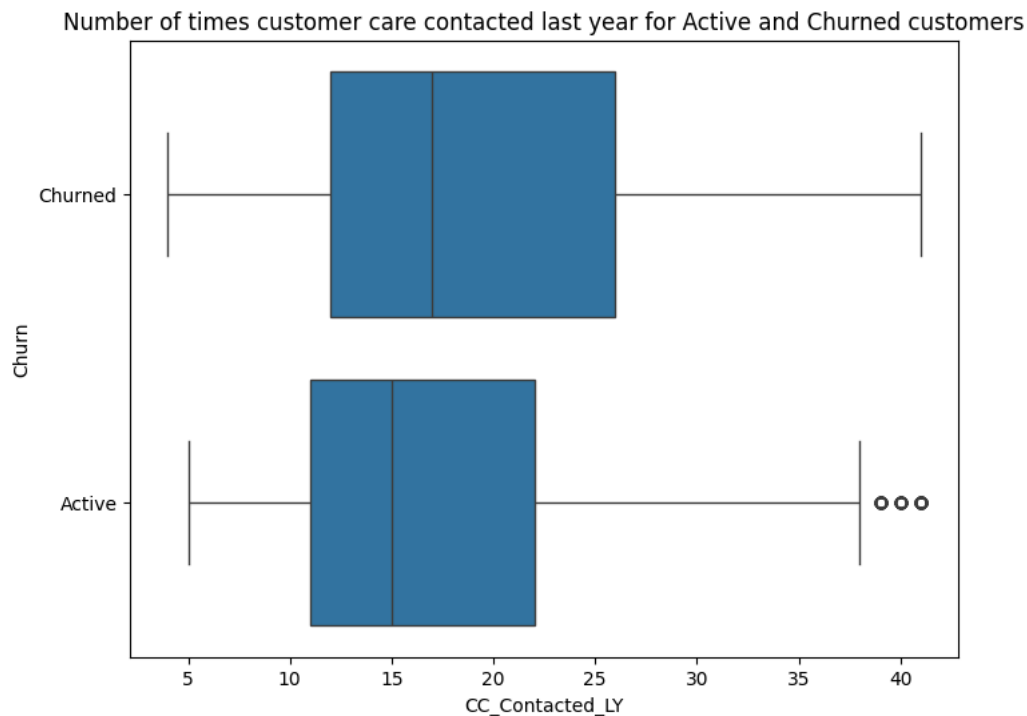
```
Index(['Tenure', 'City_Tier', 'CC_Contacted_LY', 'Payment', 'Gender',
      'Service_Score', 'Account_user_count', 'account_segment',
      'CC_Agent_Score', 'Marital_Status', 'rev_per_month', 'Complain_ly',
      'rev_growth_yoy', 'coupon_used_for_payment', 'Day_Since_CC_connect',
```

```
'cashback', 'Login_device', 'Churn', 'Tenure_bin'],
dtype='object')
```

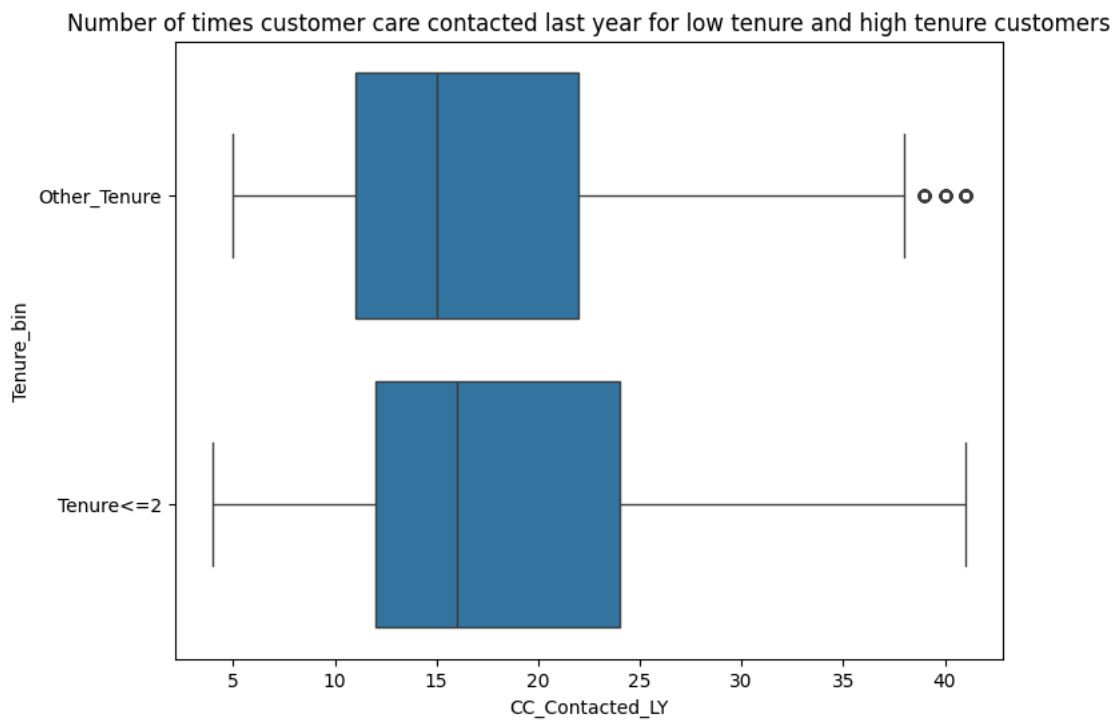
```
plt.figure(figsize=(8,6))
sns.boxplot(x=df_total["Day_Since_CC_connect"], y=df_total_churn)
plot_title = 'Boxplot of ' + "Days since CC connect" + " for Active and Churned customers"
plt.title(plot_title)
plt.show();
```



```
plt.figure(figsize=(8,6))
sns.boxplot(x=df_total["CC_Contacted_LY"], y=df_total_churn)
plot_title = 'Number of times ' + "customer care contacted last year" + " for Active and Churned customers"
plt.title(plot_title)
plt.show();
```



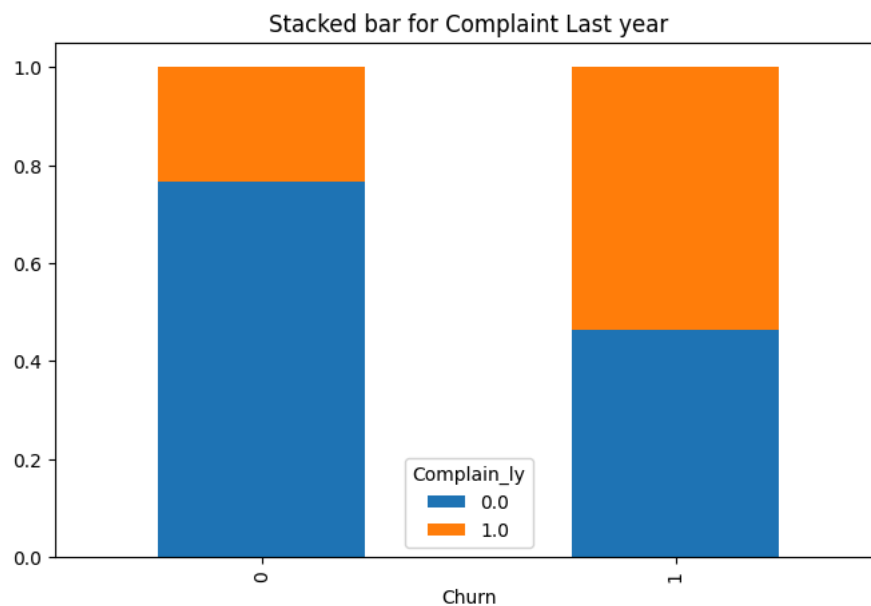
```
plt.figure(figsize=(8,6))
sns.boxplot(x=df_temp["CC_Contacted_LY"], y=df_temp['Tenure_bin'])
plot_title = 'Number of times ' + "customer care contacted last year" + " for low tenure and high tenure customers"
plt.title(plot_title)
plt.show();
```



```
tab = pd.crosstab(df_temp['Churn'],df_temp['Complain_ly'],normalize='index')
print(tab)
```

```
tab.plot(kind='bar', stacked = True, figsize=(8,5))
plot_title = "Stacked bar for " + "Complaint Last year"
plt.title(plot_title)
plt.show();
```

Complain_ly	0.0	1.0
Churn		
0	0.765592	0.234408
1	0.464135	0.535865



```
Churn_count = df_temp[df_temp['Churn']==0]['Churn'].count()
Active_count = df_temp[df_temp['Churn']==1]['Churn'].count()
```

```
Churn_count, Active_count
```

```
(np.int64(9364), np.int64(1896))
```

```
Churn_complaint = df_temp[(df_temp['Churn']==0) & (df_temp['Complain_ly']==1)]['Churn'].count()
Active_complaint = df_temp[(df_temp['Churn']==1) & (df_temp['Complain_ly']==1)]['Churn'].count()
```

```
Churn_complaint, Active_complaint
```

```
(np.int64(2195), np.int64(1016))
```

```
tab = pd.crosstab(df_temp['Churn'],df_temp['Complain_ly'],normalize='index', margins=True)
print(tab)
```

Complain_ly	0.0	1.0
Churn		
0	0.765592	0.234408
1	0.464135	0.535865
All	0.714831	0.285169

```
tab = pd.crosstab(df_temp['Complain_ly'],df_temp['Churn'],normalize='index')
print(tab)
```

Churn	0	1
Complain_ly		
0.0	0.890670	0.109330
1.0	0.683588	0.316412

```
tab.plot(kind='bar', stacked = True, figsize=(10,5))
plt.title("%Churn for customers with and without complaint")
```

```
Text(0.5, 1.0, '%Churn for customers with and without complaint')
```



Revenue per month

```
tab = pd.crosstab(df_temp['rev_per_month'],df_temp['Churn'],normalize='index', margins=False).round(2)*100
print(tab)
```

Churn	0	1
rev_per_month		
1.0	88.0	12.0
2.0	83.0	17.0
3.0	83.0	17.0
4.0	86.0	14.0
5.0	85.0	15.0
6.0	83.0	17.0
7.0	80.0	20.0
8.0	82.0	18.0
9.0	83.0	17.0
10.0	82.0	18.0
11.0	78.0	22.0
12.0	77.0	23.0
13.0	79.0	21.0

```
df_revenue_pm = pd.DataFrame(tab.astype('int64'), columns=['Active','Churned'])
```

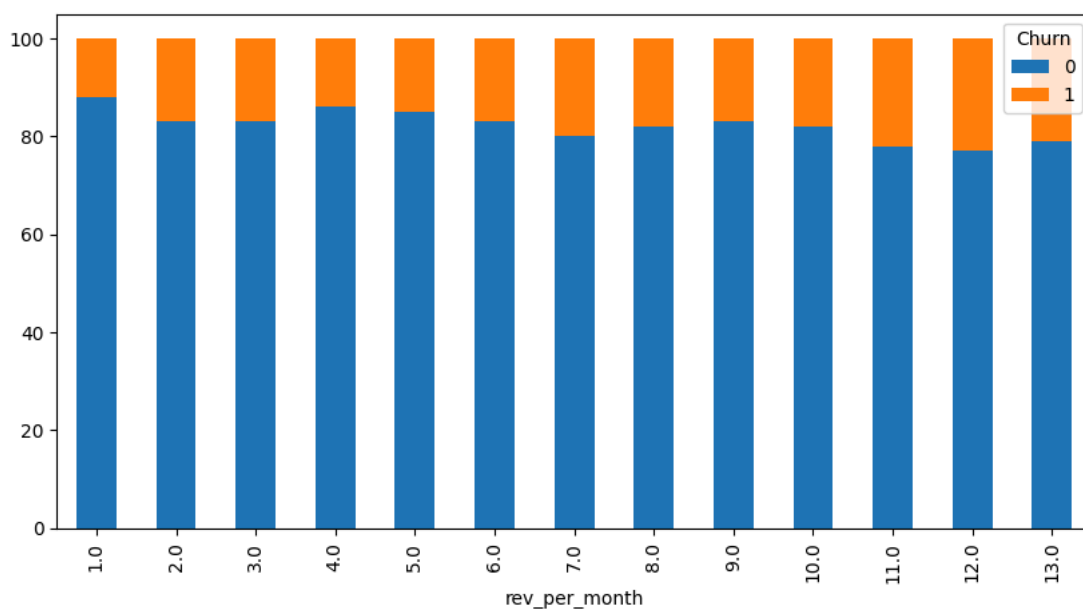
```
df_revenue_pm['Active'] = tab[0]
df_revenue_pm['Churned'] = tab[1]
```

```
df_revenue_pm
```

	Active	Churned
rev_per_month		
1.0	88.0	12.0
2.0	83.0	17.0
3.0	83.0	17.0
4.0	86.0	14.0
5.0	85.0	15.0
6.0	83.0	17.0
7.0	80.0	20.0
8.0	82.0	18.0
9.0	83.0	17.0
10.0	82.0	18.0
11.0	78.0	22.0
12.0	77.0	23.0
13.0	79.0	21.0

```
tab.plot(kind='bar', stacked = True, figsize=(10,5))
```

<Axes: xlabel='rev_per_month'>



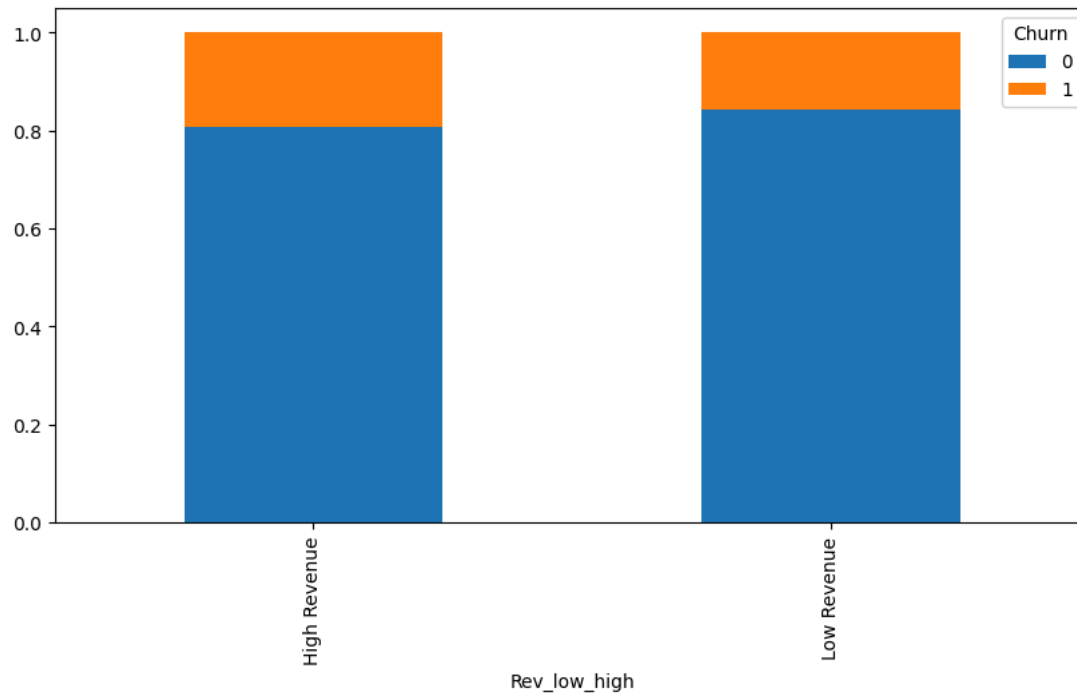
```
df_temp['Rev_low_high'] = np.where(df_temp['rev_per_month'] <= 6 , 'Low Revenue', 'High Revenue')
```

```
tab_revenue = pd.crosstab(df_temp['Rev_low_high'],df_temp['Churn'],normalize='index')
print(tab_revenue)
tab_revenue.plot(kind='bar', stacked = True, figsize=(10,5))
```

```

Churn      0      1
Rev_low_high
High Revenue  0.807214  0.192786
Low Revenue   0.841728  0.158272
<Axes: xlabel='Rev_low_high'>

```



```

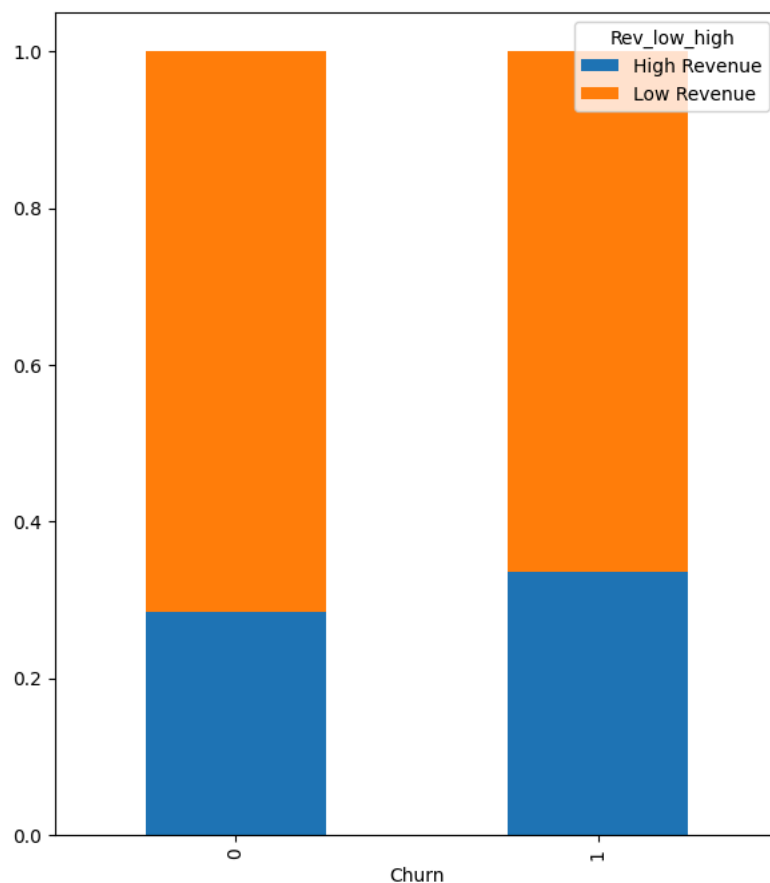
tab_revenue2 = pd.crosstab(df_temp['Churn'],df_temp['Rev_low_high'],normalize='index') #, margins=True)
print(tab_revenue2)
tab_revenue2.plot(kind='bar', stacked = True, figsize=(7,8));

```

```

Rev_low_high  High Revenue  Low Revenue
Churn
0             0.284387    0.715613
1             0.335443    0.664557

```



```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
sns.set_style("white")
```

```
df = pd.read_csv("CC_without_outlier.csv")
```

```
df.head(5)
```

	Tenure	City_Tier	CC_Contacted_LY	Service_Score	Account_user_count	CC_Agent_Score	rev_per_month	Complain_ly	Day_Since_C
0	4.0	3.0	6.0	3.0	3.0	2.0	9.0	1.0	
1	0.0	1.0	8.0	3.0	4.0	3.0	7.0	1.0	
2	0.0	1.0	30.0	2.0	4.0	3.0	6.0	1.0	
3	0.0	3.0	15.0	2.0	4.0	5.0	8.0	0.0	
4	0.0	1.0	12.0	2.0	3.0	5.0	3.0	0.0	

5 rows × 23 columns

```
df_outlier = pd.read_csv("CC_with_outlier.csv")
```

```
df_outlier.head(5)
```

	Unnamed: 0	Tenure	City_Tier	CC_Contacted_LY	Service_Score	Account_user_count	CC_Agent_Score	rev_per_month	Complain_ly	D
0	0	4.0	3.0	6.0	3.0	3.0	2.0	9.0	1.0	
1	1	0.0	1.0	8.0	3.0	4.0	3.0	7.0	1.0	
2	2	0.0	1.0	30.0	2.0	4.0	3.0	6.0	1.0	
3	3	0.0	3.0	15.0	2.0	4.0	5.0	8.0	0.0	
4	4	0.0	1.0	12.0	2.0	3.0	5.0	3.0	0.0	

5 rows × 24 columns

Machine Learning Algorithm

Logistic Regression Model - SKLearn

LR_model1 : Base model with outlier treated unscaled data

```
X = df.drop(columns=["Churn"])
y = df["Churn"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

```
LR_model1 = LogisticRegression(verbose=True, n_jobs=-1, random_state=123)
```

```
LR_model1.fit(X_train, y_train)
```

```
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
```

```
LogisticRegression
LogisticRegression(n_jobs=-1, random_state=123, verbose=True)
```

```
### Predicting on train and test dataset
```

```
LR_model1_ytrain_pred = LR_model1.predict(X_train)
LR_model1_ytest_pred = LR_model1.predict(X_test)
```

```
## Predicting probabilities on train dataset
LR_model1_trainprobs = LR_model1.predict_proba(X_train)
# # keep probabilities for the positive outcome only
LR_model1_trainprobs = LR_model1_trainprobs[:, 1]
```

```
## Predicting probabilities on test dataset
LR_model1_testprobs = LR_model1.predict_proba(X_test)
# # keep probabilities for the positive outcome only
```

```

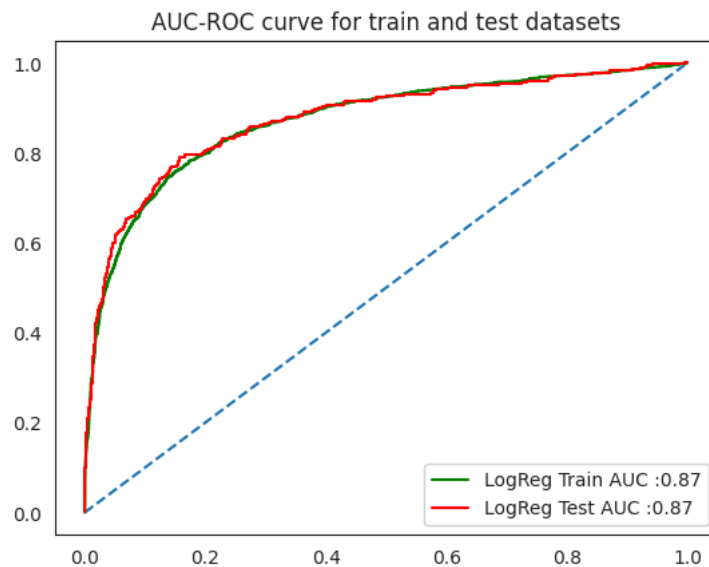
LR_model1_testprobs = LR_model1_testprobs[:, 1]

# # calculate roc curve
train_auc = round(roc_auc_score(y_train, LR_model1_trainprobs),2)
test_auc = round(roc_auc_score(y_test, LR_model1_testprobs),2)
train_label_LR = 'LogReg Train AUC :' + str(train_auc)
test_label_LR = 'LogReg Test AUC :' + str(test_auc)

fpr_train, tpr_train, thresholds_train = roc_curve(y_train, LR_model1_trainprobs)
fpr_test_LR, tpr_test_LR, thresholds_test_LR = roc_curve(y_test, LR_model1_testprobs)

# # plot the roc curve for the model
plt.plot([0, 1], [0, 1], linestyle='--')
plt.plot(fpr_train, tpr_train, color='green',label=train_label_LR)
plt.plot(fpr_test_LR, tpr_test_LR, color='red', label=test_label_LR)
plt.legend(loc='best')
plt.title("AUC-ROC curve for train and test datasets")
plt.show();

```



```

from sklearn.metrics import roc_auc_score, roc_curve, classification_report, confusion_matrix, ConfusionMatrixDisplay # Added p

def model_eval(model,xtrue,ytrue,ypred,probs):

    plot_roc_curve(model,xtrue,ytrue,color='red') # Now plot_roc_curve should be recognized.
    plt.plot([0, 1], [0, 1], linestyle='--');

    print('Accuracy : ', model.score(xtrue, ytrue))

    # Instead of plot_confusion_matrix, use ConfusionMatrixDisplay
    ConfusionMatrixDisplay.from_estimator(model, xtrue, ytrue, colorbar=False).plot();

    print(classification_report(ytrue, ypred));

    # # calculate AUC
    auc = roc_auc_score(ytrue, probs)
    print('AUC: %.3f' % auc);

```

```

from sklearn.metrics import accuracy_score
log_acc = accuracy_score(y_train, LR_model1_ytrain_pred)

```

```

print("\nModel Performance:")
print(f"Logistic Regression Accuracy: {log_acc:.4f}")
print("\nClassification Reports:")
print("\nLogistic Regression:\n", classification_report(y_train, LR_model1_ytrain_pred))

```

```

Model Performance:
Logistic Regression Accuracy: 0.8868

Classification Reports:

Logistic Regression:
      precision    recall  f1-score   support

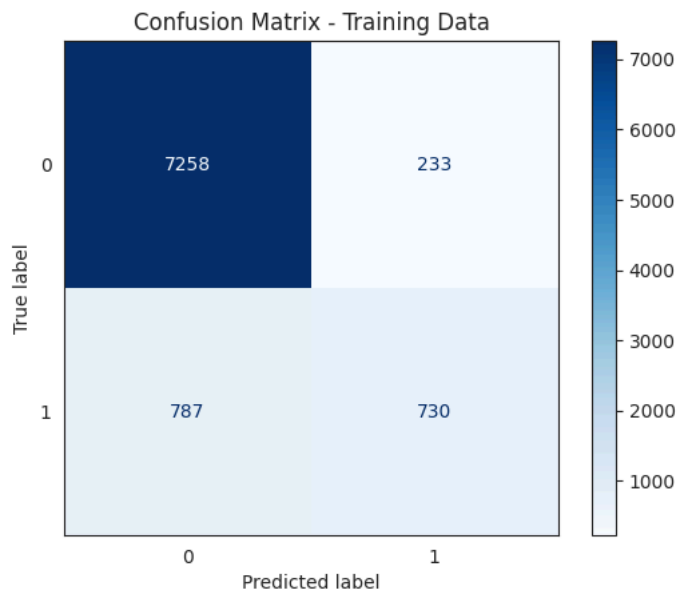
0         0.90      0.97      0.93       7491

```

	1	0.76	0.48	0.59	1517
accuracy				0.89	9008
macro avg	0.83	0.73		0.76	9008
weighted avg	0.88	0.89		0.88	9008

```
cm = confusion_matrix(y_train, LR_model1_ytrain_pred)

# Display confusion matrix
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues') # You can change the colormap if needed
plt.title("Confusion Matrix - Training Data")
plt.show()
```



```
log_acc = accuracy_score(y_test, LR_model1_ytest_pred)
```

```
print("\nModel Performance:")
print(f"Logistic Regression Accuracy: {log_acc:.4f}")
print("\nClassification Reports:")
print("\nLogistic Regression:\n", classification_report(y_test, LR_model1_ytest_pred))
```

Model Performance:
Logistic Regression Accuracy: 0.8908

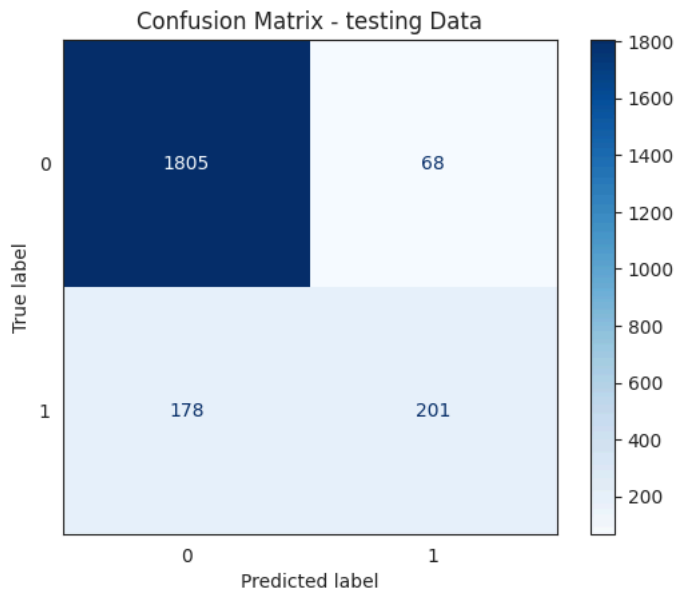
Classification Reports:

Logistic Regression:

	precision	recall	f1-score	support
0	0.91	0.96	0.94	1873
1	0.75	0.53	0.62	379
accuracy			0.89	2252
macro avg	0.83	0.75	0.78	2252
weighted avg	0.88	0.89	0.88	2252

```
cm = confusion_matrix(y_test, LR_model1_ytest_pred)

# Display confusion matrix
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues') # You can change the colormap if needed
plt.title("Confusion Matrix - testing Data")
plt.show()
```



LR_model2 : Iteration 1 - features to select = 8

```
from sklearn.feature_selection import RFE
LogR = LogisticRegression(random_state=123, max_iter=1000)
LR_model2 = RFE(estimator = LogR, n_features_to_select=8, step=1)
LR_model2 = LR_model2.fit(X_train, y_train)
```

/usr/local/lib/python3.11/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status 1): TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

LR_model2.ranking_

```
array([ 8,  1, 14, 10,  6,  7,  9,  1, 12, 15,  1,  1, 11,  4,  5,  1, 13,
        1,  1,  3,  1,  2])
```

```
selected_cols = pd.DataFrame({'Feature': X.columns, 'Rank': LR_model2.ranking_})
selected_cols[selected_cols['Rank'] == 1]
```

	Feature	Rank
1	City_Tier	1
7	Complain_ly	1
10	Payment_Credit Card	1
11	Payment_Debit Card	1
15	account_segment_Regular	1
17	account_segment_Super	1
18	account_segment_Super_Plus	1
20	Marital_Status_Single	1

```
### Predicting on train and test dataset
```

```
LR_model2_ytrain_pred = LR_model2.predict(X_train)
LR_model2_ytest_pred = LR_model2.predict(X_test)
```

```
## Predicting probabilities on train dataset
```

```
LR_model2_trainprobs = LR_model2.predict_proba(X_train)
# keep probabilities for the positive outcome only
LR_model2_trainprobs = LR_model2_trainprobs[:, 1]
```

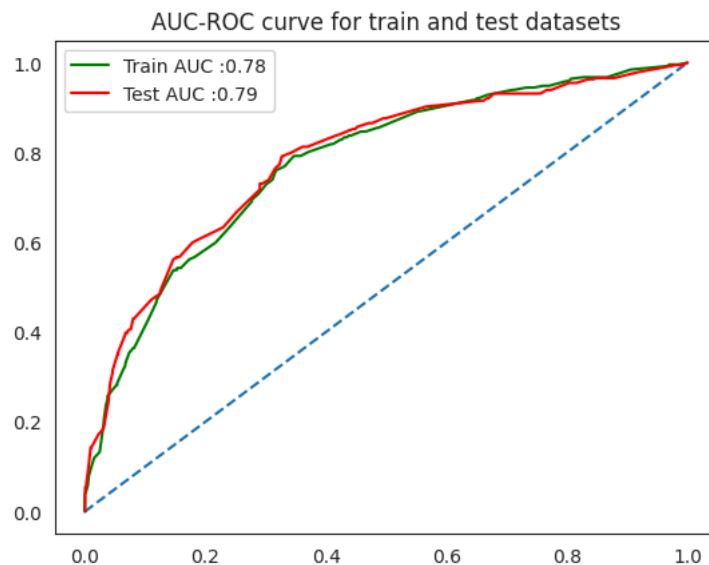
```
## Predicting probabilities on test dataset
```

```
LR_model2_testprobs = LR_model2.predict_proba(X_test)
# keep probabilities for the positive outcome only
LR_model2_testprobs = LR_model2_testprobs[:, 1]
```

```
# # calculate roc curve
train_auc = round(roc_auc_score(y_train, LR_model2_trainprobs),2)
test_auc = round(roc_auc_score(y_test, LR_model2_testprobs),2)
train_label = 'Train AUC :' + str(train_auc)
test_label = 'Test AUC :' + str(test_auc)

fpr_train, tpr_train, thresholds_train = roc_curve(y_train, LR_model2_trainprobs)
fpr_test, tpr_test, thresholds_test = roc_curve(y_test, LR_model2_testprobs)

# # plot the roc curve for the model
plt.plot([0, 1], [0, 1], linestyle='--')
plt.plot(fpr_train, tpr_train, color='green',label=train_label)
plt.plot(fpr_test, tpr_test, color='red', label=test_label)
plt.legend(loc='best')
plt.title("AUC-ROC curve for train and test datasets")
plt.show();
```



```
LogR = LogisticRegression(random_state=123, max_iter=1000)
LR_model3 = RFE(estimator = LogR, n_features_to_select=12, step=1)
LR_model3 = LR_model3.fit(X_train, y_train)
```

/usr/local/lib/python3.11/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status 1): TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
selected_cols = pd.DataFrame({'Feature': X.columns, 'Rank': LR_model3.ranking_})
selected_cols[selected_cols['Rank'] == 1]
```

	Feature	Rank
1	City_Tier	1
7	Complain_ly	1
10	Payment_Credit Card	1
11	Payment_Debit Card	1
13	Payment_UPI	1
14	Gender_Male	1
15	account_segment_Regular	1
17	account_segment_Super	1
18	account_segment_Super_Plus	1
19	Marital_Status_Married	1
20	Marital_Status_Single	1
21	Login_device_Mobile	1

```
### Predicting on train and test dataset
```

```
LR_model3_ytrain_pred = LR_model3.predict(X_train)
LR_model3_ytest_pred = LR_model3.predict(X_test)
```

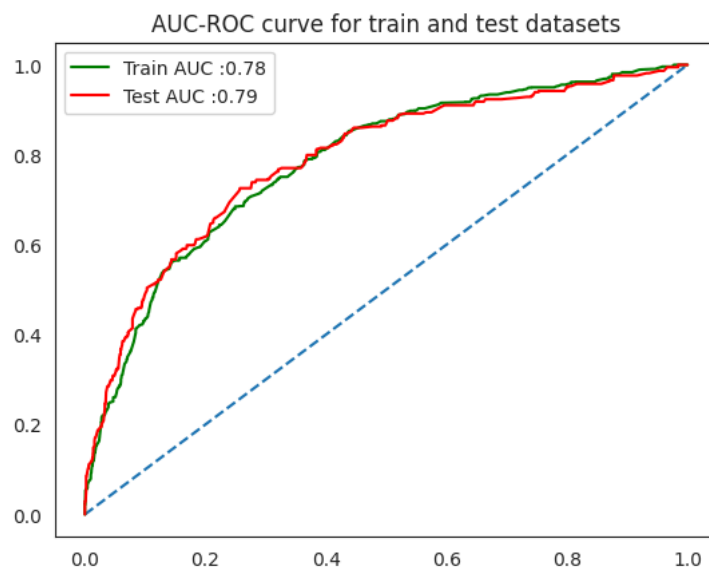
```
## Predicting probabilities on train dataset
LR_model3_trainprobs = LR_model3.predict_proba(X_train)
# # keep probabilities for the positive outcome only
LR_model3_trainprobs = LR_model3_trainprobs[:, 1]
```

```
## Predicting probabilities on test dataset
LR_model3_testprobs = LR_model3.predict_proba(X_test)
# # keep probabilities for the positive outcome only
LR_model3_testprobs = LR_model3_testprobs[:, 1]
```

```
# # calculate roc curve
train_auc = round(roc_auc_score(y_train, LR_model3_trainprobs),2)
test_auc = round(roc_auc_score(y_test, LR_model3_testprobs),2)
train_label = 'Train AUC : ' + str(train_auc)
test_label = 'Test AUC : ' + str(test_auc)
```

```
fpr_train, tpr_train, thresholds_train = roc_curve(y_train, LR_model3_trainprobs)
fpr_test, tpr_test, thresholds_test = roc_curve(y_test, LR_model3_testprobs)
```

```
# # plot the roc curve for the model
plt.plot([0, 1], [0, 1], linestyle='--')
plt.plot(fpr_train, tpr_train, color='green',label=train_label)
plt.plot(fpr_test, tpr_test, color='red', label=test_label)
plt.legend(loc='best')
plt.title("AUC-ROC curve for train and test datasets")
plt.show();
```



LR_model4 : Iteration 3 - features to select = 16

```
LogR = LogisticRegression(random_state=123, max_iter=1000)
LR_model4 = RFE(estimator = LogR, n_features_to_select=16, step=1)
LR_model4 = LR_model4.fit(X_train, y_train)
```

/usr/local/lib/python3.11/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1): TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(

```
selected_cols = pd.DataFrame({'Feature': X.columns, 'Rank': LR_model4.ranking_})
selected_cols[selected_cols['Rank'] == 1]
```

	Feature	Rank
0	Tenure	1
1	City_Tier	1
4	Account_user_count	1
5	CC_Agent_Score	1
6	rev_per_month	1
7	Complain_ly	1
10	Payment_Credit Card	1
11	Payment_Debit Card	1
13	Payment_UPI	1
14	Gender_Male	1
15	account_segment_Regular	1
17	account_segment_Super	1
18	account_segment_Super_Plus	1
19	Marital_Status_Married	1
20	Marital_Status_Single	1
21	Login_device_Mobile	1

```

### Predicting on train and test dataset

LR_model4_ytrain_pred = LR_model4.predict(X_train)
LR_model4_ytest_pred = LR_model4.predict(X_test)

## Predicting probabilities on train dataset
LR_model4_trainprobs = LR_model4.predict_proba(X_train)
# # keep probabilities for the positive outcome only
LR_model4_trainprobs = LR_model4_trainprobs[:, 1]

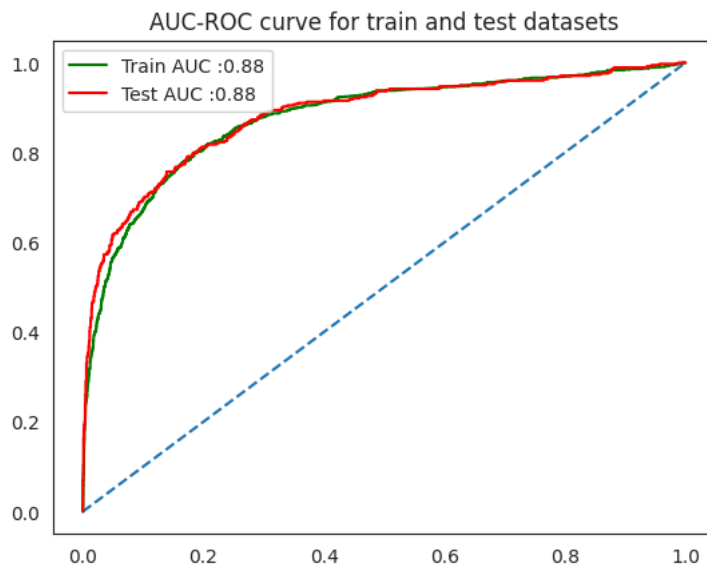
## Predicting probabilities on test dataset
LR_model4_testprobs = LR_model4.predict_proba(X_test)
# # keep probabilities for the positive outcome only
LR_model4_testprobs = LR_model4_testprobs[:, 1]

# # calculate roc curve
train_auc = round(roc_auc_score(y_train, LR_model4_trainprobs),2)
test_auc = round(roc_auc_score(y_test, LR_model4_testprobs),2)
train_label = 'Train AUC :' + str(train_auc)
test_label = 'Test AUC :' + str(test_auc)

fpr_train, tpr_train, thresholds_train = roc_curve(y_train, LR_model4_trainprobs)
fpr_test, tpr_test, thresholds_test = roc_curve(y_test, LR_model4_testprobs)

# # plot the roc curve for the model
plt.plot([0, 1], [0, 1], linestyle='--')
plt.plot(fpr_train, tpr_train, color='green',label=train_label)
plt.plot(fpr_test, tpr_test, color='red', label=test_label)
plt.legend(loc='best')
plt.title("AUC-ROC curve for train and test datasets")
plt.show();

```



Support vector machine classifier

```
from sklearn import svm
```

```

def model_eval1(model,xtrue,ytrue,ypred):

    plot_roc_curve(model,xtrue,ytrue,color='red')
    plt.plot([0, 1], [0, 1], linestyle='--');

    print('Accuracy : ', model.score(xtrue, ytrue))

    plot_confusion_matrix(model,xtrue,ytrue, colorbar=False);

    print(classification_report(ytrue, ypred));

```

```
from sklearn.preprocessing import StandardScaler
```

```

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

```

SVM_model1 : Default base model

```
## Creating model and fitting train dataset

SVM_model1 = svm.SVC(random_state=1, probability=True)
SVM_model1.fit(X_train, y_train)
```

SVC

```
SVC(probability=True, random_state=1)
```

```
### Predicting on train and test dataset
```

```
SVM_model1_ytrain_pred = SVM_model1.predict(X_train)
SVM_model1_ytest_pred = SVM_model1.predict(X_test)
```

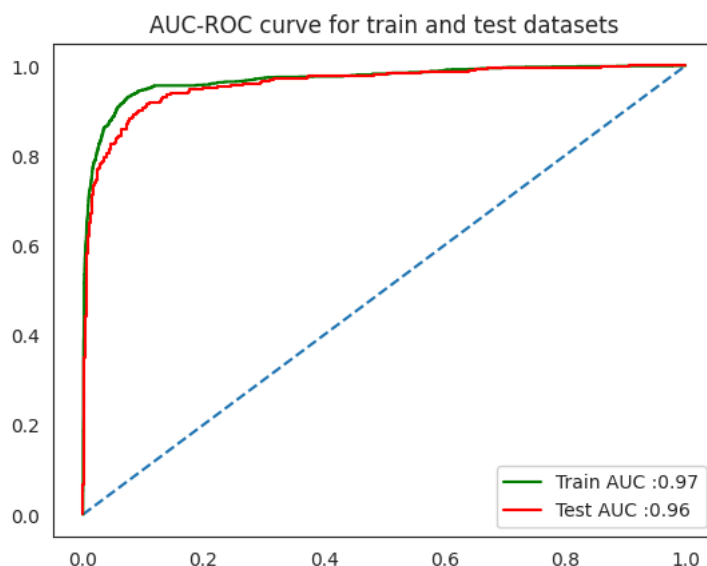
```
## Predicting probabilities on train dataset
SVM_model1_trainprobs = SVM_model1.predict_proba(X_train)
# # keep probabilities for the positive outcome only
SVM_model1_trainprobs = SVM_model1_trainprobs[:, 1]
```

```
## Predicting probabilities on test dataset
SVM_model1_testprobs = SVM_model1.predict_proba(X_test)
# # keep probabilities for the positive outcome only
SVM_model1_testprobs = SVM_model1_testprobs[:, 1]
```

```
# # calculate roc curve
train_auc = round(roc_auc_score(y_train, SVM_model1_trainprobs),2)
test_auc = round(roc_auc_score(y_test, SVM_model1_testprobs),2)
train_label = 'Train AUC : ' + str(train_auc)
test_label = 'Test AUC : ' + str(test_auc)
```

```
fpr_train, tpr_train, thresholds_train = roc_curve(y_train, SVM_model1_trainprobs)
fpr_test, tpr_test, thresholds_test = roc_curve(y_test, SVM_model1_testprobs)
```

```
# # plot the roc curve for the model
plt.plot([0, 1], [0, 1], linestyle='--')
plt.plot(fpr_train, tpr_train, color='green', label=train_label)
plt.plot(fpr_test, tpr_test, color='red', label=test_label)
plt.legend(loc='best')
plt.title("AUC-ROC curve for train and test datasets")
plt.show();
```



```
log_acc = accuracy_score(y_train, SVM_model1_ytrain_pred)

print("\nModel Performance:")
print(f"Logistic Regression Accuracy: {log_acc:.4f}")
print("\nClassification Reports:")
print("\nLogistic Regression:\n", classification_report(y_train, SVM_model1_ytrain_pred))

cm = confusion_matrix(y_train, SVM_model1_ytrain_pred)

# Display confusion matrix
```

```

disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues') # You can change the colormap if needed
plt.title("Confusion Matrix - Training Data")
plt.show()

```

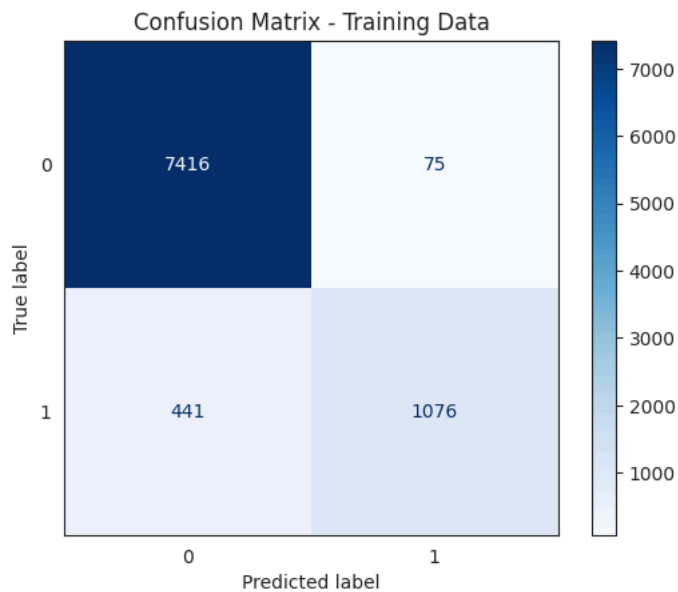
Model Performance:

Logistic Regression Accuracy: 0.9427

Classification Reports:

Logistic Regression:

	precision	recall	f1-score	support
0	0.94	0.99	0.97	7491
1	0.93	0.71	0.81	1517
accuracy			0.94	9008
macro avg	0.94	0.85	0.89	9008
weighted avg	0.94	0.94	0.94	9008



```

log_acc = accuracy_score(y_test, SVM_model1_ytest_pred)

print("\nModel Performance:")
print(f"Logistic Regression Accuracy: {log_acc:.4f}")
print("\nClassification Reports:")
print("\nLogistic Regression:\n", classification_report(y_test, SVM_model1_ytest_pred))

cm = confusion_matrix(y_test, SVM_model1_ytest_pred)

# Display confusion matrix
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
plt.title("Confusion Matrix - testing Data")
plt.show()

```

